

PHASE 6 · COMPLETE GUIDE

React + Tailwind Basics to Advanced

Build modern, professional UIs the way real apps are built — components, hooks, routing, context, custom hooks and Tailwind, from first principles.

"You already learned to update state and re-draw the screen by hand in Phase 4. React is that same idea — state → render — turned into a tool. Learn how React thinks, and every UI you'll ever build becomes just data flowing downhill."

6

DAYS

15

PARTS

62

TOPICS

Table of Contents

- **Part A — Meet React**
 - [A1. What is React & Why It Exists](#) · [A2. How React Thinks](#) · [A3. Setup: Vite & Your First Component](#)
 - **Part B — JSX: HTML in JavaScript**
 - [B1. What is JSX](#) · [B2. The JSX Rules](#) · [B3. Embedding Expressions with { }](#) · [B4. Conditional Rendering](#) · [B5. Rendering Lists & the key Prop](#)
 - **Part C — Components & Props**
 - [C1. Function Components](#) · [C2. Props — Passing Data Down](#) · [C3. Destructuring & Default Props](#) · [C4. children & Composition](#) · [C5. Thinking in Components](#)
 - **Part D — State & the useState Hook**
 - [D1. What is State \(vs Props\)](#) · [D2. useState Basics](#) · [D3. State is a Snapshot](#) · [D4. Updating Objects & Arrays Immutably](#) · [D5. Lifting State Up](#) · [D6. Controlled Forms](#)
 - **Part E — useEffect & Side Effects**
 - [E1. What is a Side Effect](#) · [E2. useEffect & the Dependency Array](#) · [E3. Cleanup Functions](#) · [E4. Fetching Data \(loading / error / success\)](#) · [E5. useEffect Gotchas](#)
 - **Part F — React Router v6**
 - [F1. Why Client-Side Routing](#) · [F2. BrowserRouter, Routes, Route](#) · [F3. Link & NavLink](#) · [F4. Dynamic Params & useParams](#) · [F5. useNavigate](#) · [F6. Nested & Layout Routes](#) · [F7. 404 & Protected Routes](#)
 - **Part G — Context & useContext**
 - [G1. The Prop-Drilling Problem](#) · [G2. createContext + Provider + useContext](#) · [G3. ThemeContext \(light/dark\)](#) · [G4. AuthContext](#) · [G5. When NOT to Use Context](#)
 - **Part H — Custom Hooks**
 - [H1. Rules of Hooks & What a Custom Hook Is](#) · [H2. useFetch](#) · [H3. useLocalStorage](#) · [H4. useDebounce](#) · [H5. useToggle & Friends](#)
 - **Part I — useReducer & Advanced State**
 - [I1. Why useReducer](#) · [I2. The reducer + dispatch Pattern](#) · [I3. useReducer vs useState](#) · [I4. Shopping Cart Example](#) · [I5. useReducer + Context = Mini-Redux](#)
 - **Part J — Performance & the Rest of the Hooks**
 - [J1. The Re-Render Mental Model](#) · [J2. useRef](#) · [J3. useMemo](#) · [J4. useCallback](#) · [J5. React.memo](#) · [J6. The Hooks Rulebook](#)
 - **Part K — Tailwind CSS**
 - [K1. What is Tailwind & Why Utility-First](#) · [K2. The Core Utilities](#) · [K3. Responsive Design](#) · [K4. Hover, Focus & Dark Mode](#) · [K5. Reusing Styles](#) · [K6. Tailwind + React Patterns](#)
 - **Part L — Master Program: The Task Tracker**
 - **Part M — Mindset: Thinking in React**
 - **Part N — Exercises & Mini-Projects**
 - **Part O — Resources & Quick Reference**
-

Part A — Meet React

Start from zero. You spent Phases 4–5 building a to-do list by hand with `save(); render();`. React is that exact pattern, industrialised. This part shows you why it exists before you write a line of it.

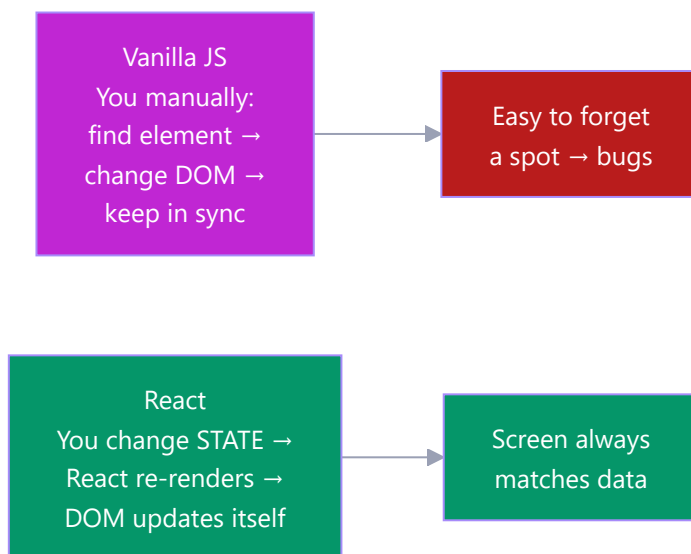
A1. What is React & Why It Exists

Simple definition: React is a JavaScript **library for building user interfaces out of small, reusable pieces called components**. You describe *what the screen should look like for a given set of data*, and React figures out *how* to update the real page to match.

Telugu: React ante UI ni **chinna chinna reusable mukkalu** (components) tho kattukune JavaScript library. Nuvvu "ee data ki screen ela undali" ani *cheptavu* (describe chestavu), React aa page ni update ela cheyyalo **tanu chusuk** ాంబుండ్రి. Phase 4 lo nuvvu `save(); render();` cheththo raasav kada — React chesedi ade pani, automatic ga. Nuvvu "ela" cheppadam maan^{సి} "emi kavali" ane cheptav.

The problem React solves. In Phase 4 you built a to-do list in vanilla JS. Every time the data changed, you manually re-drew the DOM: find the `<ul>`, clear it, loop the array, `createElement`, `append`. It worked for 40 lines. Now imagine Gmail — thousands of elements, hundreds of possible changes. Keeping the *screen* and the *data* in sync by hand becomes a nightmare of bugs: you update the array but forget to re-draw one badge; you re-draw but lose the input focus.

React removes that entire class of bug. You never touch the DOM directly. You change **state** (your data), and React re-runs your component and updates only the parts of the page that actually changed.



Where you already use it (real world):

- **Facebook, Instagram, WhatsApp Web** — React was born at Facebook.
- **Netflix, Airbnb, Uber, Swiggy** dashboards — all React.
- **SAP Fiori / UI5** — a *different* framework, but the *same ideas* (components, data binding, routing). Learning React makes UI5 click faster — that's why it's in your roadmap.

- Your **capstone FocusTrack Pro** and this phase's **Task Tracker** are React apps.

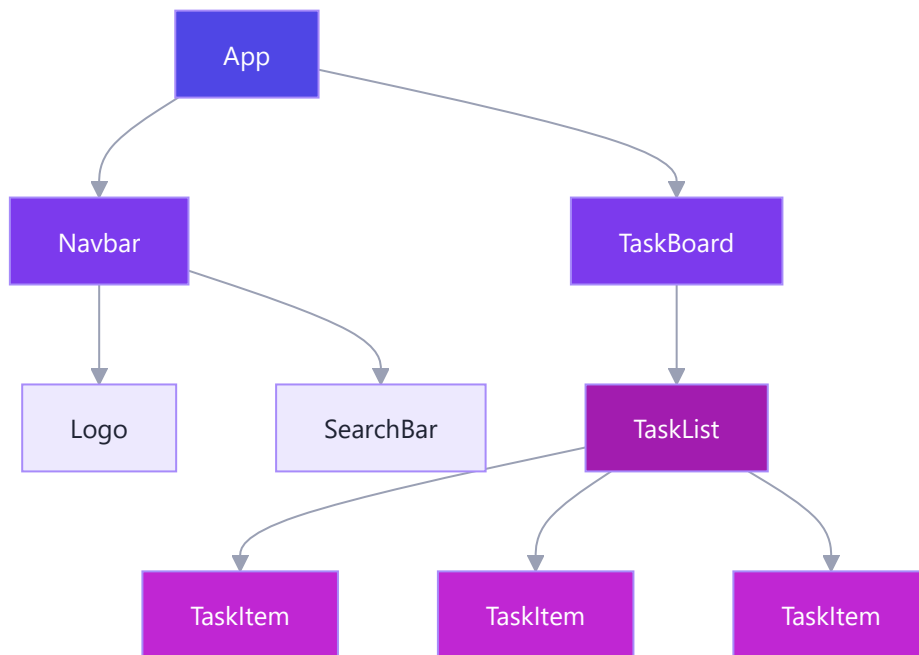
Library vs framework (a common interview line): React is a **library** — it does one thing (the UI) and leaves routing, data-fetching, etc. to other tools you add. Angular is a **framework** — it brings everything and expects you to do it its way. React's small size is why it pairs with Router, Tailwind, etc.

A2. How React Thinks — Components, Declarative, One-Way

Three ideas separate "writing React" from "writing JavaScript that happens to use React." Internalise these now and every later chapter is easy.

Telugu: React lo 3 peddha ఆలోచనలు (ideas) unnayi. (1) **Components** — UI ni chinna mukkaluga vibhajinchad (Lego blocks laga). (2) **Declarative** — "step by step ela cheyyalo" kaakunda "final ga ela undalo" cheppadam. (3) **One-way data flow** — data eppudu *paiṇuṇḍi kindiki* (parent → child) matrame prayanistundi. Ee 3 artham aithe, React antha sulabham.

1. Componentisation — build UI from Lego blocks. A page is a *tree* of components. A `<Navbar>` contains a `<Logo>` and a `<SearchBar>`; a `<TaskList>` contains many `<TaskItem>` s. Each component is a function that returns some UI. Build small pieces, then compose them.



2. Declarative, not imperative. This is the big mental shift.

- **Imperative** (vanilla JS) = *a recipe of steps*: "find the button, add a click listener, when clicked create an `<li>`, set its text, append it..."
- **Declarative** (React) = *a description of the result*: "the list shows one `<li>` per task." You state the final picture for the current data; React does the steps.

```
// Imperative (Phase 4): you list the STEPS to change the DOM
const li = document.createElement("li");
li.textContent = task.title;
list.append(li);

// Declarative (React): you describe the RESULT for the data
return <ul>{tasks.map(t => <li key={t.id}>{t.title}</li>)}</ul>;
```

3. One-way (top-down) data flow. Data travels **down** from parent to child through **props**. A child cannot reach up and change its parent's data directly. If a child needs to cause a change, the parent hands it a *function* to call. This one rule makes React apps predictable: when something on screen is wrong, the data that produced it lives in exactly one place *above* it.

Why this matters for your roadmap: SAP Fiori/UI5 uses the same MVC-ish separation and data binding. Master "data flows down, events flow up" in React and UI5's binding model will feel familiar, not foreign.

A3. Setup: Vite & Your First Component

Simple definition: **Vite** is the modern build tool that creates and runs a React project — instant startup, instant refresh when you save. It replaced the older, slower Create React App.

Telugu: **Vite** ante React project ni create chesi, run chese modern tool — chaala fast ga start avutundi, save chesina venatane browser lo kanipistundi (hot reload). Purవం "Create React App" undedీ, ippudu andaru Vite vaadathunnaru. Command okkate gurthupettuko: `npm create vite@latest`.

Create and run a project (type this once):

```
npm create vite@latest my-app -- --template react
cd my-app
npm install          # download React & friends
npm run dev          # start the dev server → http://localhost:5173
```

The folder tour — what actually matters:

File / folder	What it is
<code>index.html</code>	The single HTML page. Has one <code><div id="root"></code> — React fills it.
<code>src/main.jsx</code>	The entry point — mounts your <code><App /></code> into <code>#root</code> .
<code>src/App.jsx</code>	Your root component — where you start editing.
<code>src/</code>	All your components, hooks, and styles live here.
<code>package.json</code>	Dependencies + scripts (<code>dev</code> , <code>build</code> , <code>preview</code>).
<code>public/</code>	Static files served as-is (favicon, images).

How it boots up (main.jsx):

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";
import "./index.css";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

`createRoot(...).render(<App />)` is React grabbing that empty `#root` div and rendering your whole app into it. `<StrictMode>` is a dev-only helper that double-runs some code to surface bugs early — harmless, keep it.

Your first component (App.jsx):

```
function App() {
  const name = "Nikhil";
  return (
    <div>
      <h1>Hello, {name} 🙋</h1>
      <p>My first React component.</p>
    </div>
  );
}

export default App;
```

Read it slowly — this tiny file is all of React in miniature:

- A **component is just a function** whose name starts with a **Capital letter** (`App` , not `app` — lowercase means "HTML tag" to React).
- It **returns JSX** — HTML-looking markup (Part B).
- `{name}` drops a JavaScript value into the markup.
- `export default App` lets `main.jsx` import it.

The habit to build (same as Phase 4): keep `npm run dev` running and the browser open beside your editor. Change a word, hit save, watch it update instantly. React is learned at the keyboard — type every snippet in these notes, break it on purpose, fix it.

Part B — JSX: HTML in JavaScript

JSX is the syntax that makes React feel like writing HTML. It has a handful of rules; learn them once and 90% of "why won't this render?" errors disappear.

B1. What is JSX

Simple definition: JSX (JavaScript XML) lets you write HTML-looking markup *inside* JavaScript. It's not HTML and not a string — it's a convenient syntax that compiles to plain JavaScript function calls that create elements.

Telugu: JSX ante JavaScript **lopala HTML laaga raas** syntax. Idi nijam^{నే} HTML kaad[ం], string kaad[ం] — Vite deen[ీ]ni ven[ం] JavaScript function calls ga **maarchestundi** (compile). Enduku? UI ni chuddu^{సుంట్టు} (visual ga) ra^{య్య}chu, plus JavaScript power (variables, `.map`, if-else) anth[ం] and[ం] vaad[ం]chu. HTML + JS ok[ీ] chota — ad[ం] JSX magic.

What it compiles to — the secret behind the curtain:

```
// What you write (JSX):
const el = <h1 className="title">Hello</h1>;

// What Vite turns it into (plain JS):
const el = React.createElement("h1", { className: "title" }, "Hello");
```

You never write `createElement` yourself — but knowing JSX is *just a function call* explains every rule that follows. Because it's really JavaScript, you can store JSX in variables, return it from functions, and put it in arrays.

Why it's better than the alternatives: template strings (Phase 4's ``<li>${x}</li>``) are just text — no autocomplete, no error checking, and dangerous with user input. JSX is checked by your editor, highlights syntax, and escapes values safely by default.

B2. The JSX Rules

Six rules cover almost everything. Memorise these and JSX stops fighting you.

Telugu: JSX ki **6 rules** — ivi tel[ి] 90% errors po^{త్తాయి}. Peddha vi: (1) ok[ీ] **parent** return cheyyali, (2) HTML lo `class` ki b^{దులు} `className`, (3) anni tags **close** avali (`<img />`), (4) attributes **camelCase** (`onClick` , `tabIndex`). Gurthu: JSX = JavaScript, and[ం]ke JS ki reserve ay[ి] `class` , `for` words ni `className` , `htmlFor` ga ra^{య్య}లి.

Rule 1 — Return one root element. A component must return a *single* parent. Wrap siblings in a `<div>`, or in an empty **Fragment** `<>...</>` when you don't want an extra DOM node.

```
// ❌ two siblings at the top – error
return <h1>Hi</h1><p>Bye</p>;

// ✅ wrapped in a Fragment (adds no extra <div> to the page)
return (
  <>
    <h1>Hi</h1>
    <p>Bye</p>
  </>
);
```

Rule 2 — `className`, not `class`. `class` is a reserved word in JavaScript, so JSX uses `className`. Similarly `<label for=...>` becomes `htmlFor`.

Rule 3 — **Close every tag**. Even self-closing ones: `<img />`, `<br />`, `<input />`. HTML forgives `<img>`; JSX does not.

Rule 4 — **camelCase attributes**. `onclick` → `onClick`, `tabindex` → `tabIndex`, `stroke-width` → `strokeWidth`.

Rule 5 — `{ }` holds JavaScript. Anything in curly braces is evaluated as a JS expression (next section).

Rule 6 — `style` is an object, not a string. `style={{ color: "purple", fontSize: 20 }}` — outer braces = "JS here", inner braces = the object; CSS keys are camelCased. (In practice you'll use Tailwind classes instead — Part K.)

```
<input
  type="text"
  className="border rounded"
  onChange={handleChange}
  style={{ marginTop: 8 }}
/>
```

B3. Embedding Expressions with { }

Simple definition: curly braces `{ }` are the doorway from JSX markup back into JavaScript. Anything that *produces a value* (an **expression**) can go inside.

Telugu: `{ }` ante JSX lo "ikkada JavaScript vasthundī" ani cheppī daari. Value ni ichhī edīnā — variable, math, function call, ternary — braces lo pettīchū. Kani **statements vaadaledu**: `if`, `for`, `let` braces lo panīchēyuvu — avī value ivvavū. Value istī lopaliki, value ivvakuvōtē bayatū.

Anything that returns a value works:

```
function Profile() {
  const user = { name: "Nikhil", score: 91 };
  const now = new Date().toLocaleDateString();

  return (
    <div>
      <h2>{user.name}</h2>           {/* variable */}
      <p>Score: {user.score * 10}</p>  {/* math */}
      <p>Grade: {user.score >= 75 ? "Pass" : "Retry"}</p>  {/* ternary */}
      <p>Joined: {now}</p>             {/* function call */}
      <p>Name length: {user.name.length}</p>             {/* property */}
    </div>
  );
}
```

What does NOT work inside `{ }`: statements that don't return a value — `if`, `for`, `switch`, variable declarations. That's why React leans on **ternaries** and `.map` for logic in markup (next two sections).

```
{ if (loggedIn) { ... } } // ❌ if is a statement, not an expression
{ loggedIn ? <A/> : <B/> } // ✅ ternary IS an expression
```

How values render: strings and numbers print as-is. `true`, `false`, `null`, and `undefined` render **nothing** (this is useful — it's why `&&` works for conditionals). Objects **throw an error** — you can't render `{user}` directly, only its fields.

B4. Conditional Rendering

Simple definition: showing different UI depending on a condition. Since JSX only allows *expressions*, you use ternaries, `&&`, or early `return` s — never a bare `if` inside markup.

Telugu: Condition batti veru veru UI చూపించడం. 3 vidhaalu: (1) **ternary** `cond ? A : B` — "idi లేదా adi" ki. (2) `&&` — "unte చూపించు, లేకపోతే emi వద్దు" ki. (3) **early return** — component modat `if (loading) return ...`. Warning: `&&` ki edamchaithe 0 unte, aa 0 screen mīd kanīpistundi — andoke `count > 0 && ...` ani rayāli.

1. Ternary — "this OR that":

```
return (
  <div>
    {isLoggedIn ? <Dashboard /> : <LoginButton />}
    <span>{cart.length > 0 ? `${cart.length} items` : "Cart empty"}</span>
  </div>
);
```

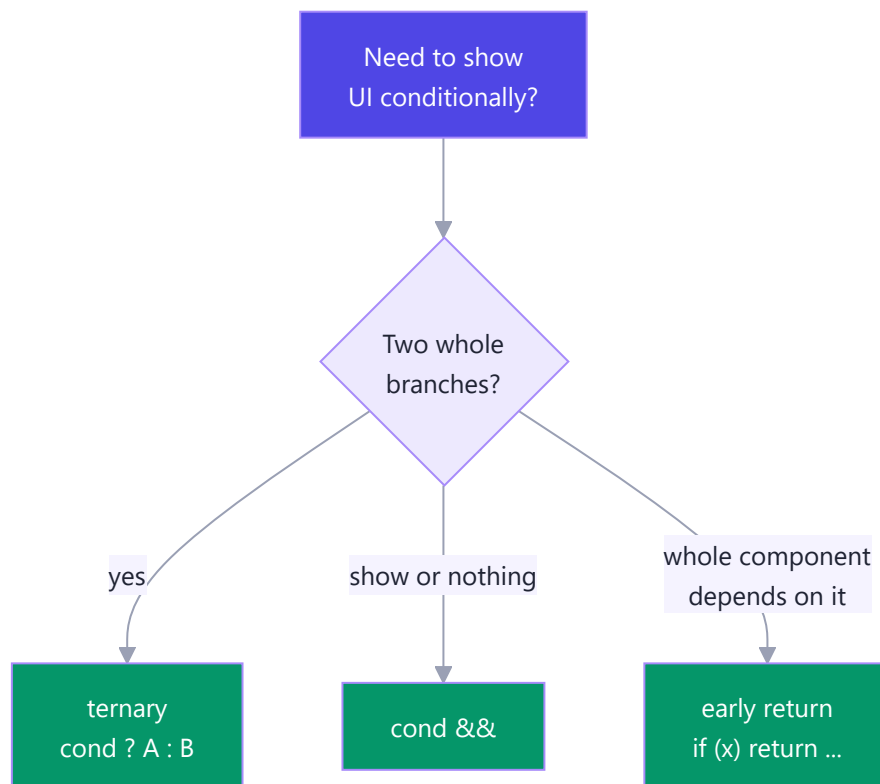
2. `&&` — "show it, or show nothing":

```
{error && <p className="text-red-600">{error}</p>}  
{unreadCount > 0 && <Badge>{unreadCount}</Badge>}
```

⚠ **The 0 trap:** `{cart.length && <Cart/>}` renders a literal 0 when the cart is empty, because 0 is falsy and renders as text. Always compare to make a real boolean: `{cart.length > 0 && <Cart/>}`.

3. Early return — for whole-component branches:

```
function UserCard({ user }) {  
  if (!user) return <p>No user selected</p>; // guard clause (Phase 4 L6!)  
  if (user.banned) return <p>Account suspended</p>;  
  
  return <div className="card">{user.name}</div>; // the happy path  
}
```



Real-world example — every data screen you'll build: loading spinner while fetching, error card if it fails, the data if it works. That's three conditions, and you'll write this shape hundreds of times (Part E4):

```
if (loading) return <Spinner />;  
if (error) return <ErrorCard message={error} />;  
return <ProductList products={products} />;
```

B5. Rendering Lists & the key Prop

Simple definition: to render an array as UI, `.map` each item to a JSX element. React requires a unique `key` prop on each so it can track items across re-renders.

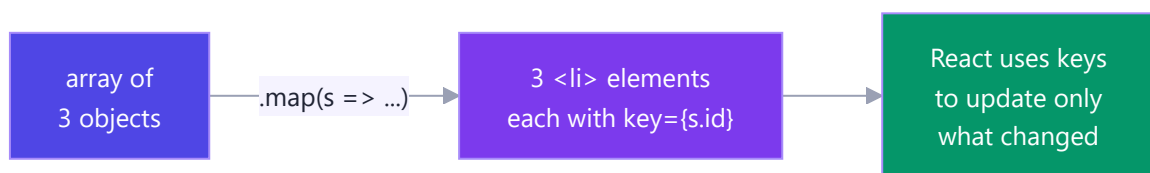
Telugu: Array ni UI ga marchadaniki `.map` వాడతాం — prati item ni okka JSX element ga maarustho. Prati element ki `key` ane unique ID ivali — React ki "edi edi" ani teliyadanisi. **Peddha rule: index ni key ga vadusu** (item delete/reorder aithi bugs); item yisthi nijaina `id` vadu. Phase 4 lo `crypto.randomUUID()` nerupukunnavu kada — akkada vadu.

The pattern — the single most-written line in React:

```
const students = [
  { id: "a1", name: "Asha", score: 91 },
  { id: "b2", name: "Bharat", score: 62 },
  { id: "c3", name: "Chitra", score: 78 },
];

function StudentList() {
  return (
    <ul>
      {students.map(s => (
        <li key={s.id}>
          {s.name} - {s.score}
        </li>
      ))}
    </ul>
  );
}
```

Why `key` matters — the identity problem. When the list changes, React compares the old and new lists to update the DOM efficiently. `key` is the stable ID that tells React "this row is the *same* row as before, just moved/edited" vs "this is a brand-new row." Without stable keys, React can attach the wrong data to the wrong DOM node — the classic bug where you delete "Asha" and "Bharat" inherits her half-typed input.



The rules of `key` :

- **Unique among siblings** (not globally — two different lists can both have `key={1}`).
- **Stable** — the same item gets the same key every render. Use the data's real `id` .
- **✗ Never the array index** when the list can reorder, filter, or delete — indexes shift and React mis-matches rows. (Index is *only* safe for a static list that never changes.)

- **✗ Never** `Math.random()` — a new key every render defeats the entire purpose (React thinks every row is new and re-creates all of them).

Real-world example: this is *literally* the JS-notes lesson `todos.map(t => t.id === id ? {...t, done:!t.done} : t)` rendered — the array is your source of truth, `.map` turns it into UI, and `key` keeps the rows honest. Filtering active-only tasks? `tasks.filter(t => !t.done).map(...)` — filter then map, exactly the Phase 4 pipeline, now producing JSX instead of DOM nodes.

Part C — Components & Props

Components are React's nouns; props are how they talk to each other. This is the vocabulary the rest of React is written in.

C1. Function Components

Simple definition: a component is a **JavaScript function that returns JSX**. Its name must start with a **capital letter**. You use it like an HTML tag: `<Button />`.

Telugu: Component ante **JSX ni return chese JavaScript function**. Peru **capital letter** tho start avvali — `Button`, `app` kaad. Enduku capital? React ki `<button>` (small) = HTML tag, `<Button>` (capital) = **నీ** component ani teలియడానికీ. **వండేటప్పుడు** HTML tag laage **వండతావు**: `<Button />`.

```
// Define it - a function returning JSX
function Welcome() {
  return <h1>Welcome back!</h1>;
}

// Use it - like a custom HTML tag
function App() {
  return (
    <div>
      <Welcome />
      <Welcome />    {/* reuse it as many times as you like */}
    </div>
  );
}
```

The capital-letter rule is not a style choice — it's how React decides:

```
<welcome />    // ✗ React thinks this is an unknown HTML tag → renders nothing useful
<Welcome />    // ✓ React calls YOUR function
```

One component per concept, one file per component (convention): `Button.jsx` exports a `Button`. Keep components small — if a function returns 200 lines of JSX, it's several components hiding in a trench coat (Phase 4 L6: "one function, one job" applies here too).

C2. Props — Passing Data Down

Simple definition: **props** (properties) are the inputs you pass *into* a component — exactly like function arguments. They flow **down** from parent to child and are **read-only**: a component can never change its own props.

Telugu: Props ante component ki ichhī **inputs** — function arguments laṁ. Parent nṁṁ child ki **kindiki matrame** prayanistṁ (one-way, gurthundṁ A2?). Peddha rule: **props read-only** — child tana props ni *maarch* lēdu. Props ni HTML attributes laage pṁṁṁṁ: `<Badge text="New" tone="warn" />`. Lopala anni okī **props** object lo vṁṁṁ.

Passing and reading props:

```
// Parent passes props (like HTML attributes)
function App() {
  return <Greeting name="Nikhil" age={26} isPro={true} />;
}

// Child receives them in a `props` object
function Greeting(props) {
  return <p>Hi {props.name}, you are {props.age}</p>;
}
```

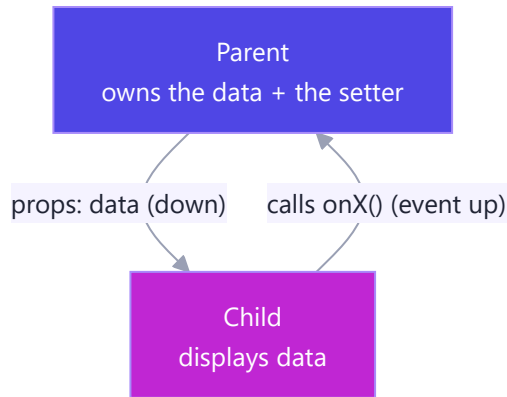
How to pass each type of value:

```
<User
  name="Nikhil"           // string → quotes
  age={26}                // number → braces
  isPro={true}            // boolean → braces
  skills={["JS", "React"]} // array → braces
  address={{ city: "BLR" }} // object → braces (double: JS + object)
  onSave={handleSave}     // function → braces (this is how events flow UP)
/>
```

Props are read-only — the golden rule:

```
function Greeting(props) {
  props.name = "changed"; // ❌ NEVER – React apps break in confusing ways
  return <p>Hi {props.name}</p>;
}
```

If a child needs to *cause* a change, the parent passes a **function** as a prop, and the child calls it. Data flows down; events (function calls) flow up. That single sentence is the whole communication model.



Real-world example — one reusable Button, many uses:

```
function Button({ label, onClick, variant }) {  
  return <button className={`btn btn-${variant}`} onClick={onClick}>{label}</button>;  
}  
  
// Used across the app with different props:  
<Button label="Save" variant="primary" onClick={save} />  
<Button label="Delete" variant="danger" onClick={remove} />
```

One definition, configured by props — the same reason functions exist in Phase 4, now for UI.

C3. Destructuring & Default Props

Simple definition: instead of writing `props.name` everywhere, **destructure** the props object right in the function's parameter list. Give defaults for props that might be missing.

Telugu: `props.name`, `props.age` ani prati saari raaykunda, **parameter loni destructure** cheyochu: `function Badge({ text, tone })`. Idi Phase 4 loni object destructuring ye (gurthund?). Missing props ki **default** ivvichu: `{ tone = "info" }`. Ee style React lo **ప్రతిచోటా** kanipistundi — andoke alavathu chesuko.

The upgrade — destructure in the parameters (the standard React style):

```
// Instead of this:
function Badge(props) {
  return <span className={props.tone}>{props.text}</span>;
}

// Write this – cleaner, and you see the "API" of the component at a glance:
function Badge({ text, tone = "info" }) { // tone defaults to "info"
  const styles = {
    info: "bg-blue-100 text-blue-700",
    warn: "bg-amber-100 text-amber-700",
    error: "bg-red-100 text-red-700",
  };
  return <span className={`px-2 py-1 rounded ${styles[tone]}`}>{text}</span>;
}

// Usage:
<Badge text="New" /> // tone falls back to "info"
<Badge text="Late" tone="warn" />
```

Why this is the norm: the destructured parameter list *documents the component* — anyone reading `function Badge({ text, tone })` instantly knows its two inputs. This is the exact "destructuring in function parameters" pattern from Phase 4 D4 — you were told then it's "the pattern React lives on." Here it is.

C4. children & Composition

Simple definition: `children` is a special prop holding whatever JSX you put *between* a component's opening and closing tags. It lets components wrap other content — the foundation of reusable layout components.

Telugu: `children` అనేది special prop — component tags `madhya` పంక్తిలోని content అంతా and లో వస్తుంది. `Id` వాడితే `<Card>...అండులో ఎంపైనా...</Card>` laga **wrapper components** కట్టొచ్చు. Analogy: `Card` ఒక photo frame; `children` లో ఏ photo (children) అనా పంక్తిలో ఉన్నా — frame ok, lopala content veru veru.

How `children` works:

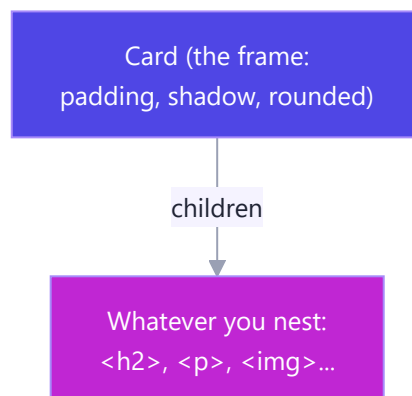
```
function Card({ children }) {
  return <div className="rounded-lg shadow p-4 bg-white">{children}</div>;
}

// Whatever you nest inside becomes `children`:
function App() {
  return (
    <Card>
      <h2>Weather</h2>
      <p>28°C, sunny</p>
    </Card>
  );
}
```

The `<h2>` and `<p>` are passed to `Card` as `children`, and `Card` decides where to place them. `Card` doesn't know or care what's inside — it just provides the frame (padding, shadow, rounded corners).

Composition beats configuration. Rather than a giant `<Card title=... body=... footer=... image=...>` with 15 props, you *compose* small pieces:

```
<Modal>
  <ModalHeader>Delete task?</ModalHeader>
  <ModalBody>This can't be undone.</ModalBody>
  <ModalFooter>
    <Button label="Cancel" />
    <Button label="Delete" variant="danger" />
  </ModalFooter>
</Modal>
```



Real-world example: every UI library (Material UI, Chakra, shadcn/ui) is built on `children` composition — `Layout`, `Card`, `Modal`, `Tooltip` all wrap your content. Your Phase 6 project's `<Layout>` (navbar + `<Outlet/>` + footer) is exactly this pattern.

C5. Thinking in Components

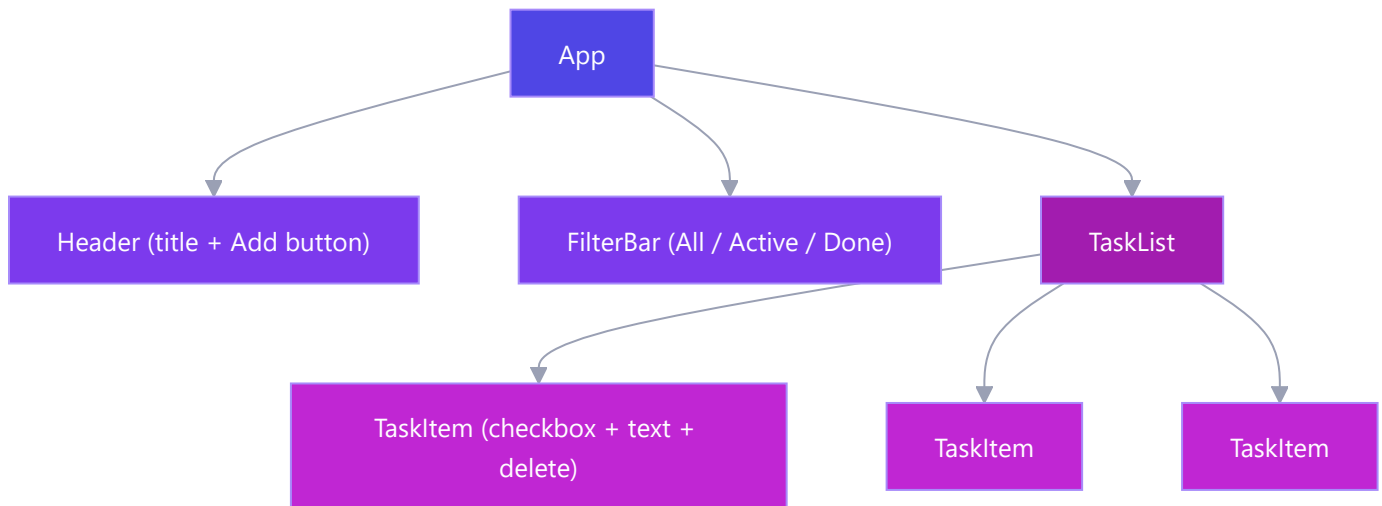
Simple definition: the design skill of looking at a UI mockup and **breaking it into a tree of components** — deciding what's a reusable piece and what's a one-off.

Telugu: Idీ ఒక **design skill** — UI ni chూసి "deenీని ఏ ఏ components ga vీభజించాలి?" ani ఆలోచించడం. Rule: **okీ paని pదే pదే kanీపేస్తే** (repeat aithీ) → adీ component. Analogy: శీర్షిక (heading), list, list-item — prati okkati ఒక chinna box. Chinna boxes kaట్టి, పెద్ద box lo pరేర్పడమే React.

The method — 3 questions for any mockup:

1. **Draw boxes around the UI.** Each box that represents *one thing* is a candidate component. A product card, a nav bar, a rating star.
2. **Spot repetition.** Anything that appears more than once with different data → definitely a component driven by props. Three product cards = one `<ProductCard>` used three times via `.map`.
3. **Arrange the tree.** Which box contains which? That containment is your component hierarchy.

Worked example — a Task Tracker page:



The two rules that guide the split:

- **Single responsibility** — a component should ideally do *one* thing. `TaskItem` shows one task; `TaskList` arranges many. (Phase 4 L6, again.)
- **Reusability** — if `<Button>` looks the same on five pages, it's one component, not five copies. Change it once, it changes everywhere.

A caution — don't over-split too early. Start with a few bigger components; extract smaller ones *when you feel the repetition or the file getting long*. Premature splitting creates a maze of tiny files. Build, notice the pattern, then refactor.

Part D — State & the useState Hook

Props are data a component receives. State is data a component owns and can change over time. This is where your UI comes alive — and it's the direct sequel to Phase 4's `save(); render();`.

D1. What is State (vs Props)

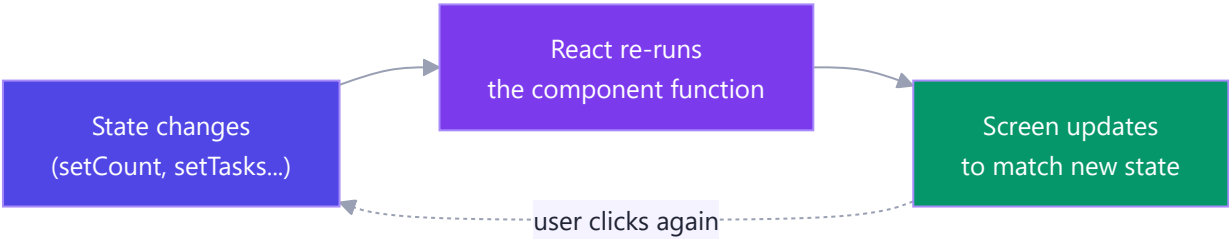
Simple definition: **state** is a component's private, changeable memory — data that can change over time due to user actions (a counter, form text, a toggle). When state changes, React **re-renders** the component to reflect it.

Telugu: State ante component **యొక్క**, **sonata, maare gala memory** — కాలంతో పాటు maarీ data (counter value, form lo type chీసిన text, dark-mode on/off). State maarీతో React aa component ni **malli gీస్తుంది** (re-render). Peddha tీడా: **Props = బయటనుంచి వస్తాయి (read-only); State = lopala పొందుతుంది, maarొచ్చు**. Phase 4 loni **tasks** array ye state; `render()` ye re-render.

Props vs State — the defining comparison:

	Props	State
Who owns it	the parent	the component itself
Can it change?	✗ read-only	✓ yes, via its setter
Comes from	outside (passed in)	inside (<code>useState</code>)
Analogy	function arguments	a function's local memory
On change	parent re-renders, passes new props	component re-renders itself

The mental model you already own. In Phase 4 your to-do list had one array `tasks` (the single source of truth) and a `render()` that redrew the DOM from it. React formalises exactly that: **state is the source of truth, and the return statement is your `render()`**. The only thing React adds is: *you don't call `render()` — changing state calls it for you.*



D2. useState Basics

Simple definition: `useState` is the **hook** that gives a component a piece of state. It returns an array of two things: the current **value**, and a **setter function** to change it.

Telugu: `useState` ante component ki state ivv^{ee} **hook**. Id^{ee} r^{ee}ndu v^{ee}stuv^{ee}la array ist^{ee}undi: [**ipp^{ee} value**, **da^{ee}ni^{ee} ma^{ee}ar^{ee} setter function**]. Aa square brackets Phase 4 loni **array destructuring** ye — `const [count, setCount] = useState(0)`. Setter ni call ch^{ee}st^{ee}ne React ki "maaru, malli g^{ee}yi" ani tel^{ee}st^{ee}undi — n^{ee}ruka `count = 5` ani ma^{ee}ar^{ee}it^{ee} pan^{ee}ch^{ee}ey^{ee}du.

Anatomy of a `useState` line:

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  //   ▲      ▲              ▲
  // value  setter    initial value

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

Read it as three parts:

- `count` — the current value, used in the JSX. Starts at `0`.
- `setCount` — the *only* correct way to change it. Calling it tells React to re-render.
- `useState(0)` — the initial value, used **only on the first render**.

The one rule that trips up every beginner — never assign directly:

```
count = count + 1;    // ❌ React has no idea anything changed → screen won't update
setCount(count + 1);  // ✅ tells React: update state AND re-render
```

Direct assignment is invisible to React. The setter is the doorbell — it says "data changed, please re-draw." (This is Phase 4's "the single doorway for change" — `setState` — now built into React.)

Multiple independent states — just call it again:

```
const [name, setName] = useState("");
const [email, setEmail] = useState("");
const [dark, setDark] = useState(false);
```

Real-world example — a live character counter:

```
function TweetBox() {
  const [text, setText] = useState("");
  const left = 280 - text.length;
  return (
    <div>
      <textarea value={text} onChange={e => setText(e.target.value)} />
      <span className={left < 0 ? "text-red-600" : ""}>{left} left</span>
    </div>
  );
}
```

Every keystroke calls `setText`, React re-renders, `left` recomputes, the number updates live. No DOM code — you describe the result, React does the rest.

D3. State is a Snapshot — The Updater Form

Simple definition: within one render, a state variable is a fixed **snapshot** — it does not change mid-function. Calling the setter schedules a *new* render with a *new* value; it doesn't mutate the current one. When the next value depends on the previous, use the **updater function** form.

Telugu: ఒక render lo state value ఒక **photo (snapshot)** లాంటిది — aa function madhyal adhi maaradhi. Setter ni call chithi kottu render schedule avuthundi, ippathi kadu. Andake baki click lo **setCount(count+1)** madhu saarlu rathu — anni oki purathana **count** ni chusthai, +3 kadu +1 ye avuthundi. Fix: **updater form** **setCount(c => c + 1)** — React naku evvathiki vundu kaja value istundi.

The classic surprise:

```
function BrokenCounter() {
  const [count, setCount] = useState(0);

  function addThree() {
    setCount(count + 1); // count is 0 → schedules 1
    setCount(count + 1); // count is STILL 0 → schedules 1
    setCount(count + 1); // count is STILL 0 → schedules 1
  } // result: 1, not 3 🤖

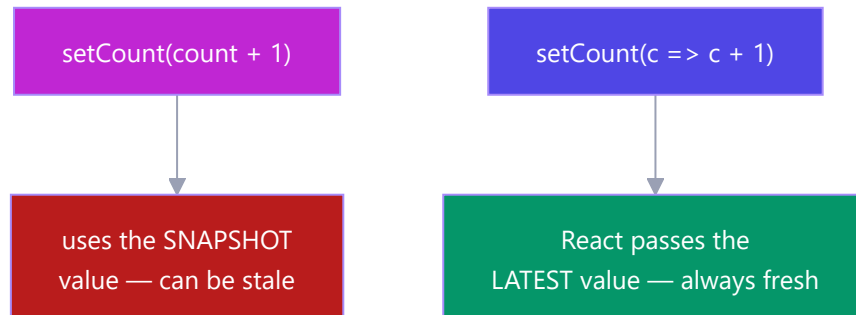
  return <button onClick={addThree}>Add 3</button>;
}
```

`count` is `0` for the entire `addThree` call — it's a snapshot. All three setters see `0`. **The fix — the updater function**, which receives the *latest* pending value:

```
function addThree() {
  setCount(c => c + 1); // c is 0 → 1
  setCount(c => c + 1); // c is 1 → 2
  setCount(c => c + 1); // c is 2 → 3 ✓
}
```

The rule: whenever the next state depends on the previous state, pass a **function** to the setter, not a value.

`setCount(count + 1)` is fine for a single independent update; `setCount(c => c + 1)` is required when they stack or when you're inside an async callback that might see stale state.



Bonus — batching: React groups multiple setter calls in the same event into **one** re-render (for performance). So even the broken version re-renders only once. Batching is why the snapshot rule exists — and why the updater form is your safety net.

D4. Updating Objects & Arrays Immutably

Simple definition: React detects change by comparing references, so you must **never mutate** state objects/arrays — you create a **new** copy with the change applied. This is Phase 4's immutability lesson, now load-bearing.

Telugu: React "state maarind?" ani **reference tho** chāṣṭundi (pureṭṇa object == kōṭṭa object? ani). Andṁke state ni **ఎప్పుడూ నీరుగా maarṁāḍadu** (mutate) — kōṭṭa copy kḷḷi, andṁlōṁ maarṁāli. `...spread` gurthundṁ Phase 4? Akkḍa nēṟṟukunnḍi ikkḍa **ప్రాణం**. `push`, `splice`, `obj.x = y` — ivi mutate chēṣṭāy, vḍḍu. `[...arr]`, `{...obj}`, `.map`, `.filter` — ivi kōṭṭa copy istṁy, ivi vḍu.

Objects — copy then override with spread:

```
const [user, setUser] = useState({ name: "Nikhil", city: "BLR" });

// ✗ mutation - React sees the SAME object reference, may not re-render
user.city = "Hyderabad";
setUser(user);

// ✓ new object with one field changed
setUser({ ...user, city: "Hyderabad" });
```

Arrays — use the non-mutating methods:

```
const [todos, setTodos] = useState([]);

// ADD:    copy + append
setTodos([...todos, newTodo]);

// REMOVE: filter keeps the rest (new array)
setTodos(todos.filter(t => t.id !== id));

// UPDATE one item: map, replace the match with a copy
setTodos(todos.map(t => t.id === id ? { ...t, done: !t.done } : t));
```

That last line — `todos.map(t => t.id === id ? { ...t, done: !t.done } : t)` — is the *exact* line the JS notes told you "is how every React list update is written." Now you know why: it produces a **new array** where one object is a modified copy and the rest pass through unchanged.

Want to...	✗ Mutating (don't)	✓ Immutable (do)
Add to array	<code>arr.push(x)</code>	<code>setArr([...arr, x])</code>
Remove from array	<code>arr.splice(i, 1)</code>	<code>setArr(arr.filter(...))</code>
Update array item	<code>arr[i].done = true</code>	<code>setArr(arr.map(...))</code>
Change object field	<code>obj.x = 1</code>	<code>setObj({ ...obj, x: 1 })</code>

⚠ **Nested objects need nested spreads** (spread is shallow — Phase 4 D5): `setUser({ ...user, address: { ...user.address, city: "Hyd" } })`. For deeply nested state, that's a signal to either restructure your state flatter or reach for `useReducer` (Part I).

D5. Lifting State Up

Simple definition: when two sibling components need to share the same data, you move that state **up** to their closest common parent, then pass it down as props. The parent becomes the single source of truth.

Telugu: రెండు sibling components okీ data ni pంచుకోవాలంటే, aa state ni వాటి **common parent** loki pైకి jarupం (lift up), taruvత props ga kindiki pంపు. Enduku? Data okీ chota (parent lo) unte, iద్దరూ okీ nీజాన్ని చంపాస్తారు — clash undదు. Analogy: iద్దరు పిల్లలు ఒక remote pంచుకోవాలంటే, remote ni tల్లి (parent) daగ్గర పెట్టి, iద్దరికీ వాడనిస్తుంది.

The problem: a `<SearchInput>` and a `<ResultsList>` are siblings. The input owns the search text, but the list needs it too. Siblings can't pass data to each other directly (data only flows down).

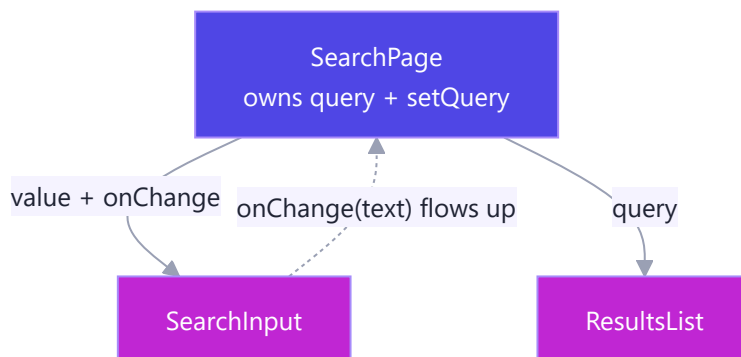
The fix — lift the state to the parent:

```
function SearchPage() {
  const [query, setQuery] = useState(""); // state lives in the PARENT

  return (
    <div>
      <SearchInput value={query} onChange={setQuery} />    {/* down: value + setter */}
      <ResultsList query={query} />                        {/* down: value */}
    </div>
  );
}

function SearchInput({ value, onChange }) {
  return <input value={value} onChange={e => onChange(e.target.value)} />;
}

function ResultsList({ query }) {
  const results = ALL.filter(item => item.includes(query));
  return <ul>{results.map(r => <li key={r}>{r}</li>)}</ul>;
}
```



The principle: state should live at the **lowest common ancestor** of every component that needs it. Not higher (unnecessary re-renders and prop-passing), not lower (siblings can't reach it). When you find yourself wanting to "send data sideways," the answer is almost always *lift it up*.

When lifting gets painful — passing the same prop through five layers to reach a deep child — that's **prop drilling**, and it's exactly the problem **Context** solves (Part G). But reach for Context only after lifting feels genuinely heavy; for two or three levels, props are simpler and clearer.

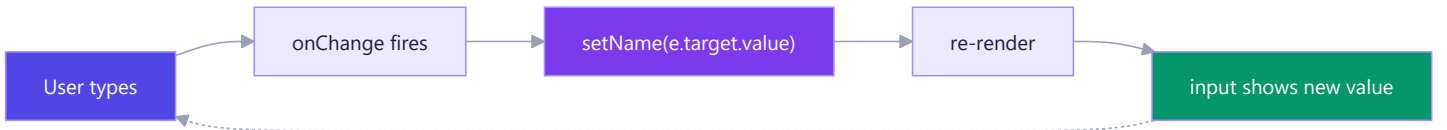
D6. Controlled Forms

Simple definition: in a **controlled input**, React state is the single source of truth for the field's value. You bind `value={state}` and update it with `onChange`. The input shows exactly what state says.

Telugu: Controlled input ante — field lo emi kanipistundi React **state ye nirbhayistundi**. Rendu tীগలు కలుపుతావు: `value={state}` (state ni field lo chupa) mariyu `onChange` (type chesht state ni update chesht). Idi loop: type → onChange → setState → re-render → field lo kott value. Andake field evvudha state tho **sync** lo untundi. Prati React form idi pattern.

The controlled input pattern:

```
function NameForm() {  
  const [name, setName] = useState("");  
  
  return (  
    <input  
      value={name} // state → input  
      onChange={e => setName(e.target.value)} // input → state  
    />  
  );  
}
```



A full form — one state object, one change handler:

```
function SignupForm() {
  const [form, setForm] = useState({ name: "", email: "", role: "user" });

  // one handler for every field, keyed by the input's `name`
  const handleChange = (e) => {
    const { name, value } = e.target;
    setForm(prev => ({ ...prev, [name]: value })); // computed key + spread
  };

  const handleSubmit = (e) => {
    e.preventDefault(); // stop the page reload (Phase 4 E3!)
    console.log(form); // { name, email, role }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" value={form.name} onChange={handleChange} />
      <input name="email" value={form.email} onChange={handleChange} />
      <select name="role" value={form.role} onChange={handleChange}>
        <option value="user">User</option>
        <option value="admin">Admin</option>
      </select>
      <button type="submit">Sign up</button>
    </form>
  );
}
```

Three things doing the heavy lifting:

- `[name]: value` — a **computed property key** (Phase 4 D3), so one handler updates whichever field fired.
- `{ ...prev, [name]: value }` — immutable object update (D4).
- `e.preventDefault()` — the same "stop the browser reloading" from Phase 4 E3.

Controlled vs uncontrolled (interview line): *controlled* = React state drives the value (what you'll use 95% of the time — it lets you validate, disable submit, transform input live). *Uncontrolled* = the DOM keeps the value and you read it with a `ref` only when needed (occasionally used for simple/large forms or file inputs). Default to controlled.

Real-world example: every login, checkout, and settings form is controlled — because state-as-truth lets you show "passwords don't match" live, disable Submit until valid (`formFields.every(f => f.value !== "")` from Phase 4 D2), and clear the form after submit with `setForm(initial)`.

Part E — useEffect & Side Effects

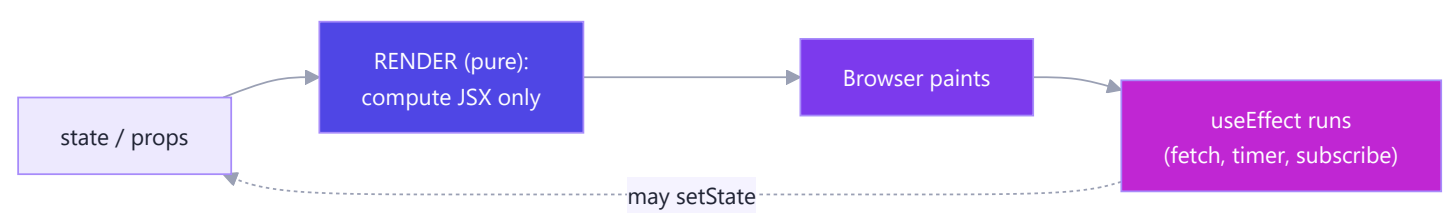
Rendering is pure — it just computes JSX from state. But real apps must also fetch data, set timers, and talk to the outside world. `useEffect` is where that "outside" work lives.

E1. What is a Side Effect

Simple definition: a **side effect** is anything a component does *besides* computing its JSX — fetching data, setting a timer, reading/writing `localStorage`, subscribing to an event, changing the document title. React calls your component to *render*; effects are for everything else.

Telugu: Side effect ante component JSX ni lekkinchadom tppa chesene migilina panulu — data fetch chesadom, timer petladom, `localStorage` lo rayadam, event ki subscribe avadam. React nee component ni **render** kheso pilustundi; render "shudham" (pure) ga undali — neruga fetch, timer akkada pettikudadu. Aa "baramiti prapanchupu" pani antho `useEffect` lo pettalali. Render = lekka; effect = baramiti pani.

Why render must stay pure. React may call your component function many times, and it expects the *same output for the same state* with no surprises. If you `fetch()` directly in the render body, it fires on every render — an infinite loop of requests. Effects give side-effect code a **controlled place and time**: *after* the render is painted to the screen.



Examples of side effects you'll write:

- Fetch data from an API when a page loads.
- Save state to `localStorage` whenever it changes.
- Set up a `setInterval` clock, or a keyboard listener.
- Sync the browser tab title to app state.

E2. useEffect & the Dependency Array

Simple definition: `useEffect(fn, deps)` runs `fn` **after render**, and re-runs it only when a value in the **dependency array** `deps` changes. The dependency array is the control knob for *when* the effect fires.

Telugu: `useEffect(fn, deps)` — `fn` ni **render taruvata** run chesundi. `deps` array ye "epudu malli run chesalo" nirbhayistundi. 3 rakulu gurthupetluko: `[]` = **ok saari** (mount lo, page vchinavvudu); `[x]` = **x maarinavvudalla**; `deps` lekavote = **prati render ki** (dondari, appadavvude avaram). Deps ni maravote bugs — andake avvudu ivva.

The three forms — this table is the whole concept:

Dependency array	When the effect runs	Use for
<code>[]</code> (empty)	once , after first render	fetch on load, set up a subscription
<code>[a, b]</code>	after first render + whenever <code>a</code> or <code>b</code> changes	re-fetch when an id/query changes
<i>(omitted)</i>	after every render	almost never — usually a bug

```
import { useState, useEffect } from "react";

function Clock() {
  const [time, setTime] = useState(new Date());

  useEffect(() => {
    const id = setInterval(() => setTime(new Date()), 1000);
    return () => clearInterval(id); // cleanup (E3)
  }, []); // [] → set the timer up ONCE

  return <p>{time.toLocaleTimeString()}</p>;
}
```

Re-run when a value changes — the most common real case:

```
function Post({ postId }) {
  const [post, setPost] = useState(null);

  useEffect(() => {
    fetch(`/api/posts/${postId}`)
      .then(r => r.json())
      .then(setPost);
  }, [postId]); // re-fetch whenever postId changes ✓

  return <article>{post?.title}</article>;
}
```

If you'd written `[]` here, switching from post 1 to post 2 would keep showing post 1 — the effect never re-runs. The dependency array `[postId]` is what keeps the data in sync with the prop.

The rule for deps: include **every** value from your component (props, state) that the effect *reads*. Leaving one out causes "stale" bugs where the effect uses an old value. (The ESLint plugin `react-hooks/exhaustive-deps` catches these for you — keep it on.)

E3. Cleanup Functions

Simple definition: if your effect sets up something ongoing (a timer, a subscription, a listener), it should **return a function** that tears it down. React runs this cleanup before the effect re-runs and when the component unmounts.

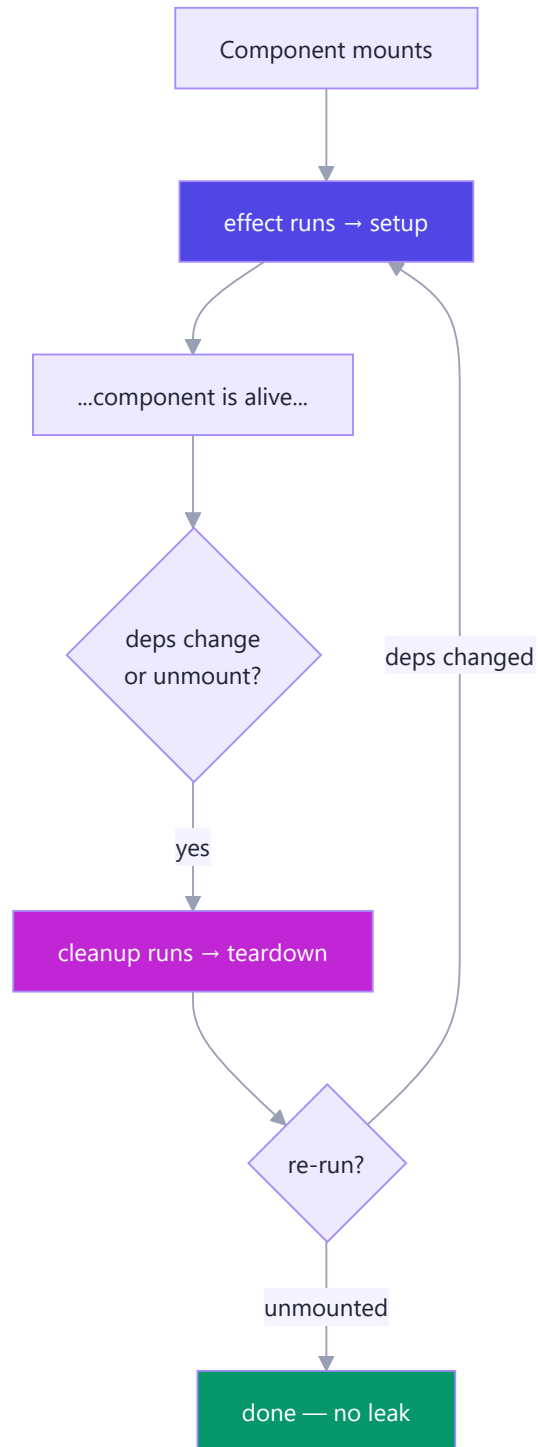
Telugu: Effect lo eదైనా **continuous paని** start చేస్తే (timer, event listener, subscription), daనిని **ఆపే function ni return** చేయాలి. React aa cleanup ni — effect malli run avvక ముందు, mariyu component పోయినప్పుడు (unmount) — run చేస్తుంది. Enduku? Marచిపోతే: timers పోరుకుపోతాయి, memory leak, పూతన listeners కూడా fire అవుతాయి. Rule: **subscribe చేస్తే, unsubscribe కూడా రాయి** — జత (pair) ga.

The shape — set up, then return the teardown:

```
useEffect(() => {
  const id = setInterval(tick, 1000); // SET UP

  return () => clearInterval(id); // CLEAN UP
}, []);
```

Why it matters — the leak it prevents. Without cleanup, every time this component re-mounts (e.g. navigating away and back) you'd stack another `setInterval`, and they'd all keep firing forever. Same with event listeners and WebSocket subscriptions. This is the *exact* lesson from Phase 4: "this is why `useEffect` returns a cleanup function — the framework forcing you to pair every subscription with an unsubscription."



Real-world cleanup examples:

```
// Event listener
useEffect(() => {
  const onKey = (e) => e.key === "Escape" && closeModal();
  window.addEventListener("keydown", onKey);
  return () => window.removeEventListener("keydown", onKey);
}, []);

// Debounced search timer (you'll turn this into useDebounce in Part H)
useEffect(() => {
  const t = setTimeout(() => runSearch(query), 400);
  return () => clearTimeout(t); // cancel the old timer on every keystroke
}, [query]);
```

Interview answer (Q5 in your roadmap): "useEffect cleanup runs (1) right before the effect re-runs due to a dependency change, and (2) when the component unmounts. In class components that second case was `componentWillUnmount`, but cleanup also handles the re-run case, which lifecycle methods didn't unify."

E4. Fetching Data (loading / error / success)

Simple definition: the canonical real-world effect — call an API on mount, and track three states: **loading**, **error**, and the **data**. This exact shape appears on nearly every screen.

Telugu: Idī React lo **ప్రతిరోజూ రాసే code** — page vచ్చినప్పుడు API పిలిచి, **3 states** చూడడం: *loading* (spinner చూపు), *error* (fail aithī message), *data* (వచ్చాక చూపు). Phase 4 loni `fetch`, `res.ok`, `try/catch/finally` anni ikkada vustai — kōttadi `useEffect` lo pōṭṭadō mariyu 3 states ni `useState` tho track chēyadō matrame.

The full pattern — memorise this skeleton:

```

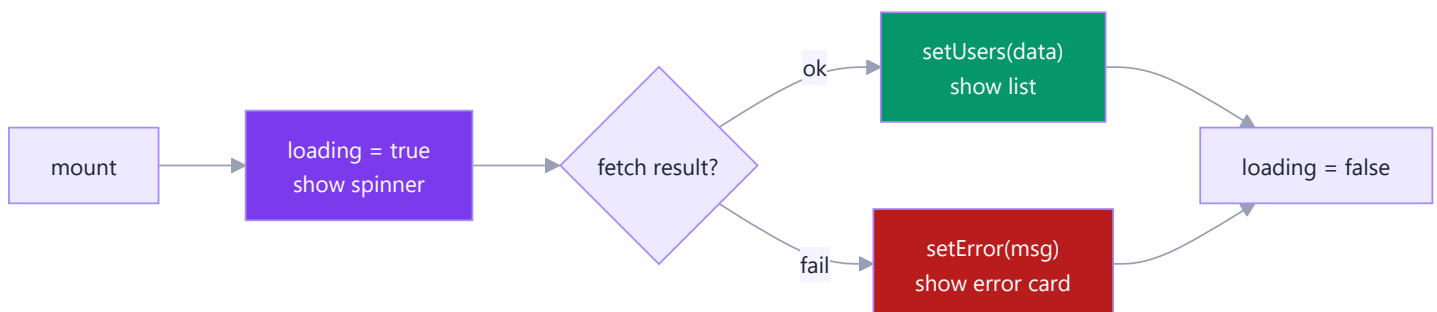
function Users() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function load() {
      try {
        setLoading(true);
        const res = await fetch("https://jsonplaceholder.typicode.com/users");
        if (!res.ok) throw new Error(`HTTP ${res.status}`); // Phase 4 H7!
        const data = await res.json();
        setUsers(data);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false); // spinner off, success OR fail
      }
    }
    load();
  }, []); // [] → fetch once on mount

  if (loading) return <p>Loading...</p>;
  if (error) return <p className="text-red-600">Error: {error}</p>;

  return (
    <ul>
      {users.map(u => <li key={u.id}>{u.name}</li>)}
    </ul>
  );
}

```



Why `async` is a nested function. An effect callback can't itself be `async` (it must return a cleanup function or nothing, not a Promise). So you define `async function load()` *inside* and call it. Remember it — it's a top-3 beginner stumble.

```
useEffect(async () => { ... }, []); // ❌ effect can't be async
useEffect(() => { async function load(){...} load(); }, []); // ✅
```

Avoiding the race-condition bug. If `postId` changes fast, an old slow response can overwrite a new one. Guard with a cleanup flag:

```
useEffect(() => {
  let active = true;
  fetch(`/api/posts/${postId}`).then(r => r.json())
    .then(data => { if (active) setPost(data); }); // ignore if superseded
  return () => { active = false; };
}, [postId]);
```

Where this goes next: this whole block is repetitive — you'll extract it into a `useFetch` custom hook in Part H2 and never write it by hand again.

E5. useEffect Gotchas You Will Hit

Simple definition: a short catalogue of the mistakes every React learner makes with effects — recognise them now and save yourself hours of debugging.

Telugu: useEffect tho andarā chāśē **common తప్పులు** — mundā tevinē guntala debugging tṇṇutundi. Peddha vāī: (1) deps array marcivōvadam → infinite loop. (2) object/function ni deps lo pṇṇṇadam → prati render kōṭṭaga → loop. (3) effect ni *event handler* avasaramaina chota vāṇṇadam. Rule: **"idī render taravāta avasaramā, līdā user click taravāta avasaramā?"** — click aithā effect vṇṇu, handler chāṇṇu.

Gotcha 1 — the infinite loop. Setting state that's in the dependency array (or with no deps) re-runs the effect forever:

```
// ❌ setCount triggers render → effect runs → setCount → render → ...
useEffect(() => { setCount(count + 1); });           // no deps → every render

// ✅ give it a reason to stop – run once, or on a specific change
useEffect(() => { setCount(c => c + 1); }, []);
```

Gotcha 2 — objects/arrays/functions as dependencies. They get a *new reference* every render, so the effect thinks they "changed" every time:

```
const options = { limit: 10 }; // new object each render
useEffect(() => { load(options); }, [options]); // ❌ runs every render

// Fix: depend on the primitive, not the object
useEffect(() => { load({ limit }); }, [limit]); // ✅
```

(When you genuinely need a stable object/function across renders, that's what `useMemo` / `useCallback` are for — Part J.)

Gotcha 3 — using an effect where an event handler belongs. Effects are for syncing with *external systems*, not for responding to a specific user action.

```
// ❌ don't POST on a state change via effect
useEffect(() => { if (submitted) api.save(form); }, [submitted]);

// ✅ just do it in the click handler
function handleSubmit() { api.save(form); }
```

The mental test before writing any effect: *"Does this need to happen because the component rendered/data changed — or because the user did a specific thing?"* The first is an effect; the second is an event handler. Modern React advice: **you might not need an effect** — reach for it only for genuine outside-world sync (fetching, subscriptions, non-React widgets).

Gotcha 4 — StrictMode double-run. In development, `<StrictMode>` runs each effect **twice** on mount to help you catch missing cleanups. That's not a bug — if your effect breaks when run twice, it's missing a cleanup. It runs once in production.

Part F — React Router v6

A React app is one HTML page. Router makes it feel like many pages — changing the URL and swapping components without a full reload. This is how every real React app has more than one screen.

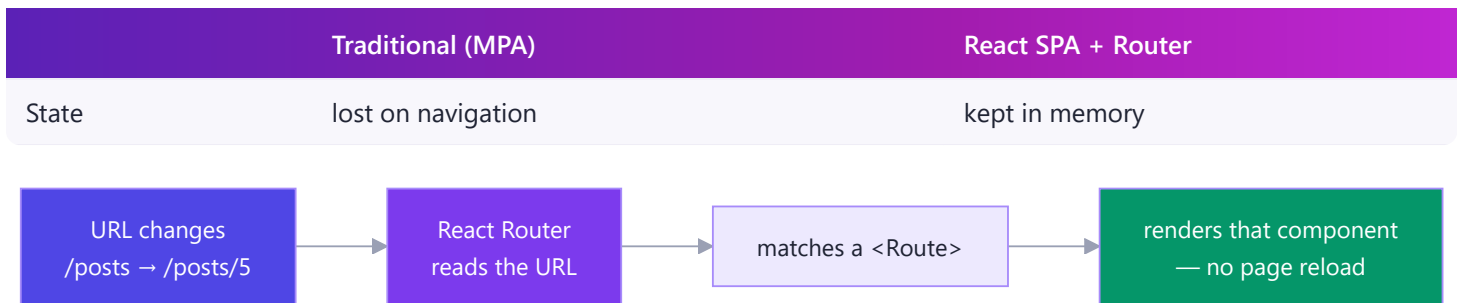
F1. Why Client-Side Routing

Simple definition: React apps are **Single-Page Applications (SPAs)** — one `index.html`. **Client-side routing** watches the URL and swaps which component is shown, with no server round-trip and no white flash. React Router is the library that does this.

Telugu: React app anedi **ok HTML page** (Single-Page App). Kani manaku /home, /about, /posts la^ondi chala "pages" kavali. **Client-side routing** ante — URL ni ch^oasⁱ, e component ch^oas^alo^o React t^one maar^ostundi, server ki m^ol^oi v^ol^ol^oundi, page reload l^oundi. And^oke chala **fast** — ok^o v^ol^ol^o page maar^ostundi. React Router aa pa^oni ch^one library.

MPA vs SPA — what changed:

	Traditional (MPA)	React SPA + Router
Each link click	server sends a new HTML page	JS swaps the component
Page reload	yes — white flash	no — instant
Speed	slower (full round-trip)	fast (already loaded)



Install it:

```
npm install react-router-dom
```

Real-world context: Gmail, Twitter, your Task Tracker — click around and the URL changes, the content changes, but the page never fully reloads. That's client-side routing. (SAP Fiori's routing does the same job for UI5 apps — another concept that transfers.)

F2. BrowserRouter, Routes, Route

Simple definition: three components set up routing. `<BrowserRouter>` wraps your app and watches the URL. `<Routes>` holds all your routes. Each `<Route path element>` maps a URL pattern to a component.

Telugu: మూడు components tho routing set చేస్తాం. `<BrowserRouter>` = మొత్తం app ni చూట్టి URL ni గమనిస్తుంది (okీ saari, top lo). `<Routes>` = anni routes ni పోర్ట్ చేద్దా dabbā. `<Route path="..." element={...} />` = "ee URL వస్తే, ee component చూపు" ane rule. Simple: path = URL, element = ఏ component.

The setup — wrap once, then declare routes:

```
import { BrowserRouter, Routes, Route } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/posts" element={<Posts />} />
        <Route path="/posts/:id" element={<PostDetail />} />
        <Route path="*" element={<NotFound />} />    {/* 404 catch-all */}
      </Routes>
    </BrowserRouter>
  );
}
```

Read it as a lookup table: React Router looks at the current URL and renders the `element` of the first `<Route>` whose `path` matches.

URL	Renders
/	<Home />
/about	<About />
/posts	<Posts />
/posts/42	<PostDetail /> (with id = "42")
/anything-else	<NotFound /> (the * wildcard)

<BrowserRouter> goes at the top, once — usually wrapping `<App />` in `main.jsx`, or as the outermost element of `App`. Everything that uses routing (links, params) must be *inside* it.

F3. Link & NavLink

Simple definition: `<Link to="/path">` is Router's replacement for `<a href>`. It navigates **without reloading the page**. `<NavLink>` is a `Link` that also knows when it's the *active* route, so you can style the current tab.

Telugu: Normal `<a href>` వాడకూడదు — అదీ మొత్తం page reload చేస్తుంది, SPA benefit పోతుంది. బాదులు `<Link to="/about">` వాడు — reload లేకుండా navigate చేస్తుంది. `<NavLink>` ante Link + "నేను ఇప్పుడు active ā?" ane teలీవి — active tab ni highlight చేయడానికి వాడతాం (navbar lo current page ni చూపడానికి).

Link — the reload-free navigation:

```
import { Link } from "react-router-dom";

<nav>
  <Link to="/">Home</Link>
  <Link to="/posts">Posts</Link>
  <Link to="/about">About</Link>
</nav>
```

⚠ Using `<a href="/posts">` instead would trigger a **full page reload** — losing all state and defeating the SPA. Always `<Link>` for internal navigation; `<a>` only for external sites.

NavLink — auto-styling the active tab:

```
import { NavLink } from "react-router-dom";

<NavLink
  to="/posts"
  className={({ isActive }) => isActive ? "font-bold text-indigo-600" : "text-slate-600"}
>
  Posts
</NavLink>
```

`NavLink` passes an `{ isActive }` flag to a `className` function, so the current page's link styles itself differently — exactly how the highlighted tab in every navbar works.

F4. Dynamic Params & `useParams`

Simple definition: a **dynamic segment** like `/posts/:id` matches any value in that slot. The `useParams()` hook reads that value inside the component, so one `<Route>` serves infinitely many URLs.

Telugu: `/posts/:id` lo `:id` ante "ee chota ఏ value అనా match అవుతుంది" — `/posts/1`, `/posts/2`, `/posts/999` anni okీ route. Component lopala aa value ni `useParams()` tho chదువుతాం: `const { id } = useParams()`. Enduku peddha విషయం? okీ route tho **వీల posts** చాపొచ్చు — prati daనికీ veru page raయక్కర్లేదు.

Define the pattern, read the value:

```
// In App: the route with a dynamic :id
<Route path="/posts/:id" element={<PostDetail />} />

// Inside PostDetail: read the id from the URL
import { useParams } from "react-router-dom";

function PostDetail() {
  const { id } = useParams();          // /posts/42 → id = "42" (always a string)

  const [post, setPost] = useState(null);
  useEffect(() => {
    fetch(`/api/posts/${id}`).then(r => r.json()).then(setPost);
  }, [id]);                          // re-fetch when the URL id changes (E2!)

  if (!post) return <p>Loading...</p>;
  return <article><h1>{post.title}</h1><p>{post.body}</p></article>;
}
```



The list → detail pattern (the most common route pair):

```
// Posts list: each item links to its own detail URL
function Posts({ posts }) {
  return posts.map(p => (
    <Link key={p.id} to={` /posts/${p.id}`}>{p.title}</Link>
  ));
}
```

Click a post → URL becomes `/posts/7` → `PostDetail` reads `id = "7"` → fetches and shows that post. One route, every post. (Your roadmap's Day 3 practice is exactly this: Home / Posts list / Post detail.)

F5. `useNavigate` — Programmatic Navigation

Simple definition: `useNavigate()` returns a `navigate` function that changes the URL from **code** — for when navigation happens after an action, not a click on a link (e.g. after login, after saving, a "Back" button).

Telugu: App డెవలపర్లు link click కాదు, **code** ని `navigate` చేయాలి — login అన తర్వాత `/dashboard` కి పంపడం, form save అన తర్వాత `mళ్ళీ` list కి, లేదా "Back" button. అందుకు `useNavigate()` — id ని `navigate` function ఇస్తుంది: `navigate("/dashboard")`. Back కి `navigate(-1)`. Link = user click; navigate = code decision.

Navigate from an event handler:

```
import { useNavigate } from "react-router-dom";

function LoginForm() {
  const navigate = useNavigate();

  async function handleLogin(credentials) {
    await api.login(credentials);
    navigate("/dashboard"); // go there after successful login
  }
  // ...
}
```

Common uses:

```
navigate("/posts"); // go to a path
navigate("/posts", { replace: true }); // replace history (no "back" to login)
navigate(-1); // go BACK one step (like the browser back button)
navigate(1); // go forward
```

Link vs navigate — the rule: if the user *clicks something that looks like a link/button to go somewhere*, use `<Link>`. If navigation is a *consequence of logic* (login succeeded, form saved, session expired → redirect to `/login`), use `navigate()`.

F6. Nested & Layout Routes — Outlet

Simple definition: routes can nest inside a **layout route** — a parent component (shared navbar/sidebar/footer) that renders its matched child via the `<Outlet />` placeholder. Shared chrome is written once.

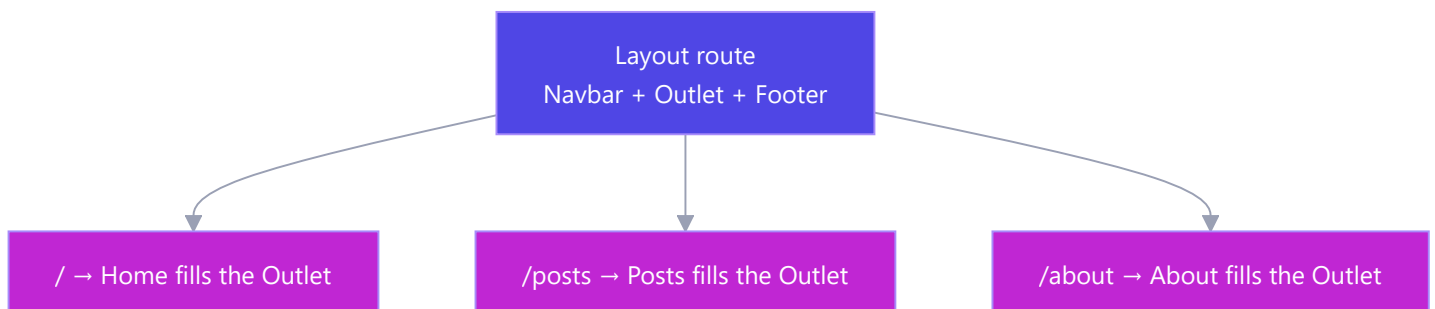
Telugu: Chలా pages lo okీ navbar, sidebar, footer repeat avutాయి. Prati page lo malli raaykunda, **layout route** vada — okీ parent (navbar + footer) lī, maare content ni `<Outlet />` ane chota pāda. Router aa Outlet chota current child page ni pāduthundi. Analogy: `Outlet` = photo frame lo **ఖాళీ చోటు**; ఏ page vā aa photo akkada vāstundi, frame (navbar/footer) okī.

A layout wrapping many pages:

```
import { Outlet } from "react-router-dom";

function Layout() {
  return (
    <div>
      <Navbar />
      <main>
        <Outlet />      {/* the matched child route renders HERE */}
      </main>
      <Footer />
    </div>
  );
}

// Nest child routes inside the layout route:
<Routes>
  <Route element={<Layout />}          {/* no path → pure layout */}
    <Route path="/"      element={<Home />} />
    <Route path="/posts" element={<Posts />} />
    <Route path="/about" element={<About />} />
  </Route>
  <Route path="/login" element={<Login />} />  {/* outside the layout – no navbar */}
</Routes>
```



Now `Home`, `Posts`, and `About` all show inside the same navbar/footer without repeating that markup. `Login` sits outside, so it renders full-screen with no navbar — perfect for auth pages.

F7. 404 & Protected Routes

Simple definition: the `*` wildcard route catches any unmatched URL (your 404 page). A **protected route** is a wrapper that checks auth and either shows the page or redirects to `/login`.

Telugu: 404: ఏ route ā match కాలేకపోతే, `path="*" (wildcard)` aa "page దొరకలేదు" ni చూపుతుంది — ఎప్పుడూ చివరివరకు వెళ్తుంది. **Protected route:** login కాని వాళ్లు /dashboard చూడకూడదు — andoake ఒక wrapper: "logged in ā? ayithe page chూపు; kాలేకపోతే `<Navigate to='/login' />` tho poncheyi." Idi nī roadmap Q4 — gurthంచుకో.

404 — always last, catches everything else:

```
<Route path="*" element={<NotFound />} />
```

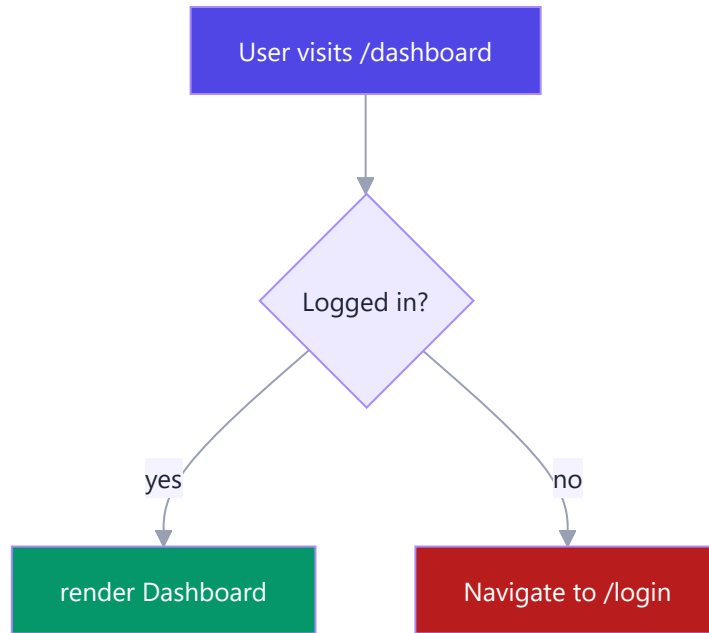
Because Router matches top-to-bottom, `*` at the end only fires when nothing above matched.

Protected route — the redirect-if-not-authed guard:

```
import { Navigate } from "react-router-dom";

function ProtectedRoute({ children }) {
  const { user } = useAuth(); // from AuthContext (Part G4)
  if (!user) return <Navigate to="/login" replace />; // kick out guests
  return children; // logged in → show the page
}

// Wrap any route you want to guard:
<Route
  path="/dashboard"
  element={
    <ProtectedRoute>
      <Dashboard />
    </ProtectedRoute>
  }
/>
```



<Navigate> vs useNavigate : **<Navigate>** is a *component* you render to redirect immediately during render (perfect for guards). **useNavigate** is a *function* you call from event handlers (F5). Guard = **<Navigate>** ; button click = **navigate()** .

Real-world example: this is every app's auth flow — dashboards, settings, and checkout are wrapped in **<ProtectedRoute>** ; the ***** route shows a friendly "Page not found" with a link home. Both are on your Day 3 / Q4 checklist.

Part G — Context & useContext

Props flow down one level at a time. When data needs to reach a deep child through many layers that don't care about it, Context lets you broadcast it directly. This is how themes, auth, and language settings travel.

G1. The Prop-Drilling Problem

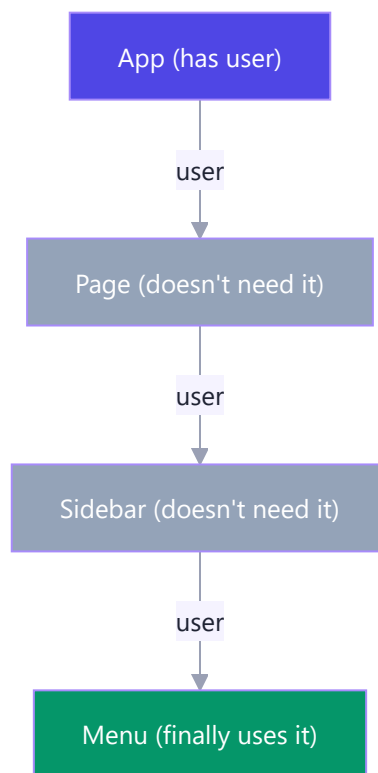
Simple definition: prop drilling is passing a prop down through many intermediate components that don't use it themselves — just to reach a deep child. It clutters every layer and makes refactoring painful.

Telugu: Prop drilling ante — ఒక data ni చాలా layers గుండా కిందికి పంపడం, madhyal unna components ki aa data avasaram lēkōyina, kēvalam kīndi child ki chērchadānisi. App → Page → Sidebar → Menu → Button ki **user** pampalante, madhyal anni "postman" laga mōyalī. Idi cōraku (annoying), plus okk prop maaritē chālā files touch chēyalī. Context idi fix chēstundi.

The problem in code — **user** passed through components that don't need it:

```
function App() {
  const [user] = useState({ name: "Nikhil" });
  return <Page user={user} />;           // pass down...
}
function Page({ user }) {
  return <Sidebar user={user} />;       // ...just to pass down...
}
function Sidebar({ user }) {
  return <Menu user={user} />;         // ...just to pass down...
}
function Menu({ user }) {
  return <span>Hi {user.name}</span>;    // FINALLY used, 3 layers deep
}
```

`Page` and `Sidebar` don't use `user` at all — they're forced to accept and forward it. Multiply this by theme, language, and auth, and every component's props become noise.



When it's *not* a problem: passing a prop one or two levels is normal and clear — don't reach for Context to avoid a single hop. Prop drilling becomes worth solving at ~3+ layers, or when *many* components need the same global-ish value.

G2. createContext + Provider + useContext

Simple definition: Context has three parts. `createContext()` makes a context object. A `<Context.Provider value={...}>` broadcasts a value to everything inside it. `useContext(Context)` reads that value from any depth — no props in between.

Telugu: Context ki 3 **bhāgālu**: (1) `createContext()` — ఒక "channel" create చేయడం. (2) `<Provider value={...}>` — aa channel లో value ni **broadcast** చేయడం (radio station laga), lopala unna andarīkī andoతుంది. (3) `useContext(Ctx)` — ఏ depth loనైనా aa value ni **నేరుగా andంకోవడం** (radio on చేసినట్టు), madhya props అక్కర్లేదు. Broadcast okS chota, vīnadam ఎక్కడైనా.

The three steps:

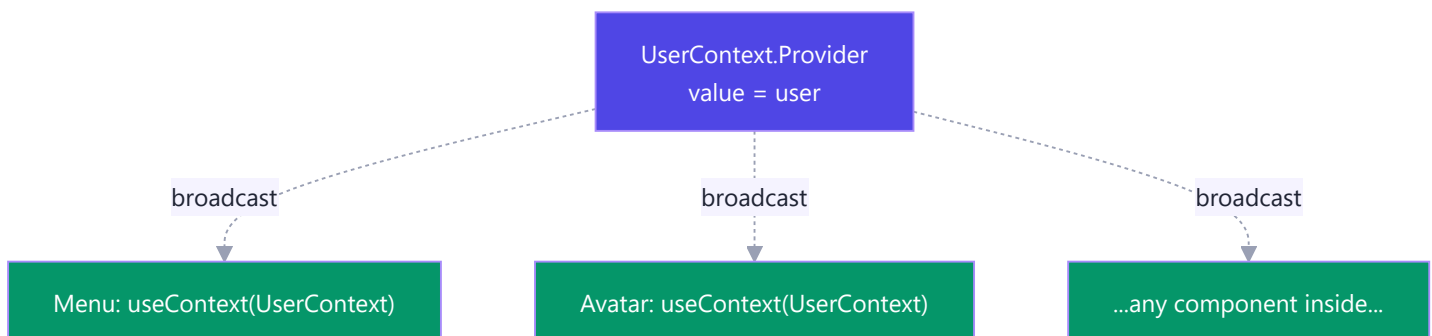
```
import { createContext, useContext, useState } from "react";

// 1. CREATE the context (usually in its own file)
const UserContext = createContext(null);

// 2. PROVIDE a value at the top - everything inside can read it
function App() {
  const [user, setUser] = useState({ name: "Nikhil" });
  return (
    <UserContext.Provider value={user}>
      <Page />           {/* no user prop needed anywhere below! */}
    </UserContext.Provider>
  );
}

// 3. CONSUME it anywhere below, at any depth
function Menu() {
  const user = useContext(UserContext); // reach in directly
  return <span>Hi {user.name}</span>;
}
```

`Page` and `Sidebar` disappear from the data path entirely — no more forwarding. `Menu` grabs `user` straight from the context.



The standard pattern — bundle state + a custom hook in the context file:

```
// useContext.jsx
const UserContext = createContext(null);

export function UserProvider({ children }) {
  const [user, setUser] = useState(null);
  const login = (u) => setUser(u);
  const logout = () => setUser(null);
  return (
    <UserContext.Provider value={{ user, login, logout }}>
      {children}
    </UserContext.Provider>
  );
}

// a tiny custom hook so consumers write useUser() not useContext(UserContext)
export const useUser = () => useContext(UserContext);
```

This bundles the value *and* the updater functions into one context, and exposes a clean `useUser()` hook — the professional pattern you'll copy for Theme and Auth next.

G3. ThemeContext (light/dark)

Simple definition: a real, complete example — a light/dark theme that any component can read and toggle, with zero prop drilling. This is your Phase 6 Day 4 deliverable.

Telugu: Nija example — light/dark theme ni ఏ component నైనా చదవగలదు, toggle చేయగలదు, props లోకుండా. Provider lo `theme` ("light"/"dark") + `toggle` function పోట్టి broadcast చేస్తాం. Navbar lo button `toggle()` పీల్చేస్తే, మొత్తం app theme maareతుంది. Tailwind lo `dark:` classes tho idీ కలిస్తే (Part K4), professional dark mode readyగా ఉంటుంది.

The complete ThemeContext:

```
// ThemeContext.jsx
import { createContext, useContext, useState } from "react";

const ThemeContext = createContext();

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState("light");
  const toggle = () => setTheme(t => (t === "light" ? "dark" : "light"));

  return (
    <ThemeContext.Provider value={{ theme, toggle }}>
      {children}
    </ThemeContext.Provider>
  );
}

export const useTheme = () => useContext(ThemeContext);
```

Wrap the app, then use it anywhere:

```
// main.jsx or App.jsx
<ThemeProvider>
  <App />
</ThemeProvider>

// Any component – a toggle button, no props passed
function ThemeToggle() {
  const { theme, toggle } = useTheme();
  return (
    <button onClick={toggle}>
      {theme === "light" ? "🌙 Dark" : "☀️ Light"}
    </button>
  );
}

// Another component reads the same theme to style itself
function Page() {
  const { theme } = useTheme();
  return <div className={theme === "dark" ? "bg-slate-900 text-white" : "bg-white"}>...</div>;
}
```

One source of truth (`theme`), broadcast to all. The button toggles it, every subscriber re-renders with the new value. That's the whole feature — and it's why "global-ish UI state" is Context's sweet spot.

G4. AuthContext

Simple definition: the same pattern for the logged-in user. `AuthContext` holds `user`, `login`, and `logout`, so any component — navbar, protected route, profile page — can read auth status directly.

Telugu: Adī pattern, ippondu **login aṁś user** kōṣoṁ. `AuthContext` iṁ `user`, `login`, `logout` pēḍatāo. Navbar "logout" chāpālā? — `useAuth()`. Protected route "login aṁśyārā?" ani adṁālā (Part F7)? — `useAuth()`. Profile page user peru kōṣālā? — `useAuth()`. Okī chota auth state, app antho vāḍukuntundi.

A mock AuthContext (real one just swaps in an API call):

```
// AuthContext.jsx
import { createContext, useContext, useState } from "react";

const AuthContext = createContext();

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null); // null = logged out

  const login = (name) => setUser({ name }); // real app: await api.login()
  const logout = () => setUser(null);

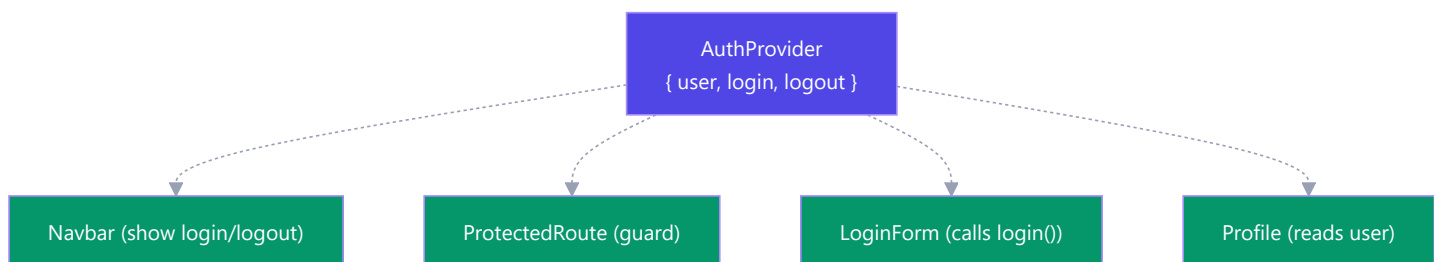
  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => useContext(AuthContext);
```

How the whole app uses it:

```
// Navbar shows login state
function Navbar() {
  const { user, logout } = useAuth();
  return user
    ? <button onClick={logout}>Logout ({user.name})</button>
    : <Link to="/login">Login</Link>;
}

// ProtectedRoute (from F7) reads the same context
function ProtectedRoute({ children }) {
  const { user } = useAuth();
  return user ? children : <Navigate to="/login" replace />;
}
```



Provider nesting. Real apps wrap multiple providers — order doesn't matter unless one depends on another:

```
<AuthProvider>
  <ThemeProvider>
    <App />
  </ThemeProvider>
</AuthProvider>
```

G5. When NOT to Use Context

Simple definition: Context is a tool with a cost — every consumer re-renders when the value changes, and it can make components harder to reuse. Use it for genuinely global, rarely-changing data; not as a dumping ground for all state.

Telugu: Context **anni da**నికీ **వాడకూడదు**. Cost undi: value maari the aa context ni **prati component** re-render avutundi. Rule: **అరుదుగా maare, app-anta avasaramaina** data ke Context — theme, auth, language. Rindu components madhya data aithe **lift up** chalu (D5). Chala tarchuna maare data (form typing laanti) Context lo pottaku — nammudistundi.

Good fits for Context (global, low-frequency change):

- Theme (light/dark), language/locale.
- Authenticated user / session.

- App-wide settings, feature flags.

Poor fits — use something else:

Situation	Don't	Do
Two nearby components share data	Context	lift state up (D5)
Fast-changing value (input text)	Context (re-renders everyone)	local <code>useState</code>
Complex server data + caching	hand-rolled Context	a data library (React Query) — <i>awareness</i>
Deep but simple prop passing	Context for everything	just pass the prop 1–2 levels

The performance gotcha: because *all* consumers re-render when the Provider's `value` changes, putting rapidly-changing state in a widely-used context causes app-wide re-renders. Keep contexts focused (a separate `ThemeContext` and `AuthContext`, not one giant `AppContext`), and for heavy shared logic, pair Context with `useReducer` (Part I5) rather than sprinkling `useState` everywhere.

Rule of thumb: reach for Context when you catch yourself drilling the *same* prop through *many* layers to *many* leaves. Otherwise, props and lifting are simpler — and simple wins.

Part H — Custom Hooks

A custom hook is how you extract and reuse stateful logic. Once you can write one, repetitive patterns like "fetch with loading/error" or "sync to localStorage" become one-line imports. This is React's real superpower.

H1. Rules of Hooks & What a Custom Hook Is

Simple definition: a **custom hook** is a JavaScript function whose name starts with `use` and that calls other hooks (`useState`, `useEffect`, ...). It bundles reusable *stateful logic* so multiple components can share it — without sharing state.

Telugu: Custom hook ante — peru `use` tho start avē, lopala vērē hooks (`useState`, `useEffect`) vāḍē JavaScript function. Idī **repeat aye logic** ni okī chota pōḍṭī, chālā components lo vāḍukōṣṭānisi. Muḃyāḃ: idī *logic* ni share chēṣṭundi, *state* ni kōḍu — prati component ki tana sonta copy vṣṭundi. `useFetch`, `useLocalStorage` — ivi nāvṣṣā rāṣē tools.

Why extract a hook? Say three components each fetch data with the same loading/error `useState` + `useEffect` block (Part E4). That's copy-paste. A custom hook `useFetch(url)` holds that logic *once*; each component calls it and gets its *own* independent state.

```
// Before: same 15 lines in every component
// After:
const { data, loading, error } = useFetch("/api/users");
```

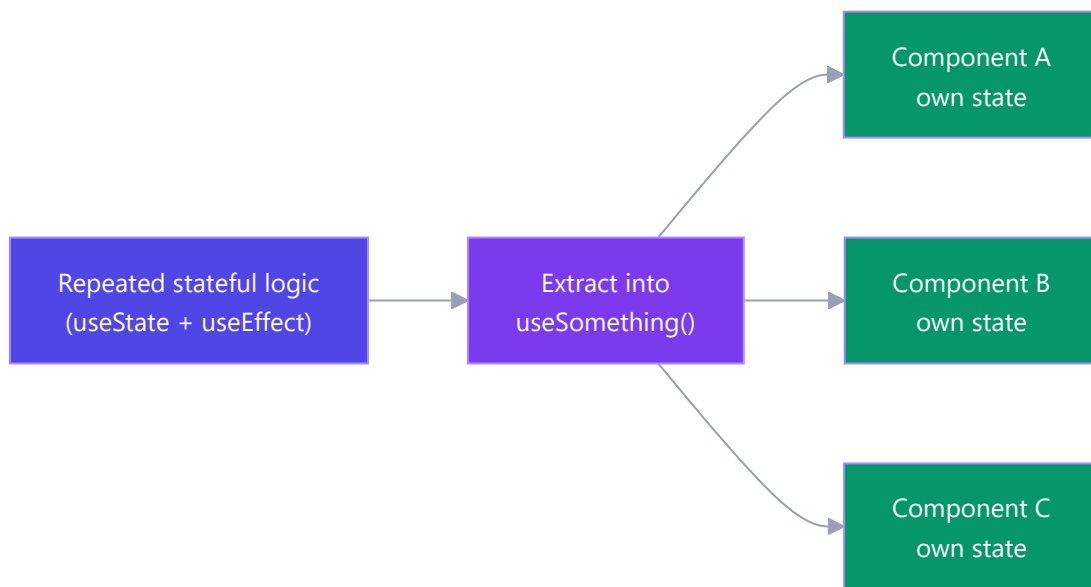
The Rules of Hooks — two rules, never break them:

1. **Only call hooks at the top level** — never inside `if`, loops, or nested functions. React tracks hooks by call *order*; a conditional hook throws that order off.
2. **Only call hooks from React functions** — components or other custom hooks. Not from regular functions or event handlers.

```
// ❌ breaks Rule 1 - hook inside a condition
if (loggedIn) {
  const [x, setX] = useState(0);
}

// ✅ hook at top level; put the condition INSIDE
const [x, setX] = useState(0);
if (loggedIn) { /* use x */ }
```

Why the `use` prefix is mandatory: it's how React's linter *knows* a function is a hook and must obey the rules. Name it `useToggle`, not `toggle`. (The `react-hooks` ESLint plugin enforces both rules — keep it on and it catches mistakes as you type.)



H2. useFetch

Simple definition: a custom hook that wraps the loading/error/data fetching pattern from E4 into one reusable line. Every data-fetching component becomes three lines shorter.

Telugu: Part E4 Ioni fetch pattern (loading/error/data) ni okś hook lo pēḥṭadō. Ippudu prati component ī aa 15 lines rayyākkōdēdu — `const { data, loading, error } = useFetch(url)` okś line chālū. URL maaritē malli fetch avatundi (deps lo url). Idi nī roadmap Q1/Q2 tools ī okḷī — okṣari rāni, everywhere vādū.

The hook — the E4 pattern, extracted:

```
// useFetch.js
import { useState, useEffect } from "react";

export function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let active = true; // race-condition guard (E4)
    setLoading(true);
    setError(null);

    fetch(url)
      .then(res => {
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        return res.json();
      })
      .then(json => { if (active) setData(json); })
      .catch(err => { if (active) setError(err.message); })
      .finally(() => { if (active) setLoading(false); });

    return () => { active = false; }; // ignore stale responses
  }, [url]); // re-fetch when the URL changes

  return { data, loading, error };
}
```

Using it — the payoff:

```
function Users() {
  const { data: users, loading, error } = useFetch("/api/users");

  if (loading) return <Spinner />;
  if (error) return <ErrorCard message={error} />;
  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}

// A totally different component - same hook, its own state:
function Products() {
  const { data: products, loading } = useFetch("/api/products");
  // ...
}
```

Two components, one hook, **independent** state each. That's the whole point — reuse the *logic*, not the *data*. This is the biggest "aha" of React: your app becomes a set of small, composable hooks.

H3. useLocalStorage

Simple definition: a `useState` that also **persists to** `localStorage` — so the value survives a page refresh. Same API as `useState`, but the data is saved automatically. (Roadmap Q2.)

Telugu: `useState` లాగే, కానీ value ని `localStorage` లో **కాపాడే** `save` చేస్తుంది — refresh అనా పోదు. Phase 4 లోని `JSON.stringify / JSON.parse` + `localStorage` జత గుర్తుందా? Adీ ikkడ hook ga. వాడకో `useState` లాగే: `const [name, setName] = useLocalStorage("name", "")`. Value మారినప్పుడల్లా automatic ga save. To-do apps, theme, cart — anni deenీత్ persist avutాయి.

The hook — `useState` + auto-save:

```
// useLocalStorage.js
import { useState, useEffect } from "react";

export function useLocalStorage(key, initialValue) {
  // read once on init - lazy initializer runs only on first render
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key);
    return saved !== null ? JSON.parse(saved) : initialValue;
  });

  // save whenever the value (or key) changes
  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue]; // same shape as useState
}
```

Using it — a drop-in `useState` replacement:

```
function App() {
  const [tasks, setTasks] = useLocalStorage("tasks", []);
  const [theme, setTheme] = useLocalStorage("theme", "light");
  // tasks and theme now survive refresh - no extra code
}
```

Two details worth noticing:

- The **lazy initializer** `useState(() => ...)` — passing a *function* to `useState` runs it only on the first render, so you don't hit `localStorage` on every render. (A neat pattern beyond this hook.)

- It returns `[value, setValue]` — the *same tuple shape* as `useState` — so it's a true drop-in replacement. Good hook design mimics familiar APIs.

Real-world example: your Task Tracker's tasks persist with this exact hook (roadmap deliverable). Theme, cart, "dismissed this banner", draft form text — all one-liners now. This is Phase 4's JSON + localStorage lesson, promoted to a reusable React tool.

H4. useDebounce

Simple definition: debouncing delays reacting to a fast-changing value until it stops changing for a moment.

`useDebounce` returns a value that only updates after the input has been quiet for `delay` ms — perfect for search boxes that shouldn't fire an API call on every keystroke. (Roadmap Q1.)

Telugu: Debounce ante — తరచుగా maare value ki, adiki ఆగే varaku wait chēyadō. Search box lo prati letter ki API call chēstē, "react" type chēstē 5 calls — waste + slow. `useDebounce` value ni user ఆగినట్టు అనిపించే varaku (say 400ms) update chēyadu. Aipaina oki call. Phase 4 loni `setTimeout` / `clearTimeout` + cleanup ye idi. Live search ki mandatory.

The hook — a self-cancelling timer:

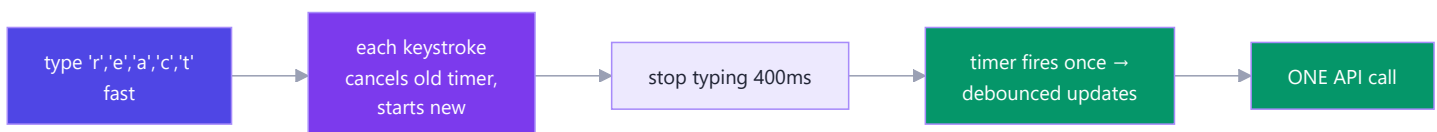
```
// useDebounce.js
import { useState, useEffect } from "react";

export function useDebounce(value, delay = 300) {
  const [debounced, setDebounced] = useState(value);

  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(timer); // cancel if value changes before delay
  }, [value, delay]);

  return debounced;
}
```

How it works — the cleanup is the trick. Every keystroke changes `value`, which re-runs the effect. The cleanup **cancels the previous timer** before starting a new one. Only when you *stop* typing for `delay` ms does a timer survive and fire, updating `debounced`.



Using it — debounced live search:

```
function Search() {
  const [query, setQuery] = useState("");
  const debouncedQuery = useDebounce(query, 500); // waits for a pause

  const { data } = useFetch(
    debouncedQuery ? `/api/search?q=${debouncedQuery}` : null
  );

  return (
    <div>
      <input value={query} onChange={e => setQuery(e.target.value)} />
      { /* input updates instantly; API only fires 500ms after you stop */ }
    </div>
  );
}
```

The input stays snappy (uses `query`), but the API only sees `debouncedQuery`. Typing "react" fires **one** request, not five. This composes two of your custom hooks — `useDebounce` feeding `useFetch` — which is exactly how hooks are meant to stack.

H5. useToggle & Friends

Simple definition: small utility hooks for the tiny patterns you repeat constantly — toggling a boolean, tracking the previous value, copying to clipboard. They make components read like plain English.

Telugu: Chinna chinna repeat aye patterns ki chinna hooks. `useToggle` — boolean ని on/off చేయడం (modal open/close, menu show/hide). Prati saari `setOpen(!open)` raaykunda, `toggle()` chalu. Ivi chesinnave kani, code ni **english laga** chadivelu cheshtayi. Nuvvu nee sona hooks library kappuvadam ikkada madalvutundi.

useToggle — cleaner boolean state:

```
// useToggle.js
import { useState, useCallback } from "react";

export function useToggle(initial = false) {
  const [on, setOn] = useState(initial);
  const toggle = useCallback(() => setOn(o => !o), []); // updater form (D3)
  return [on, toggle, setOn];
}

// Usage - reads like English:
function Modal() {
  const [isOpen, toggleOpen] = useToggle(false);
  return (
    <>
      <button onClick={toggleOpen}>{isOpen ? "Close" : "Open"}</button>
      {isOpen && <div className="modal">...</div>}
    </>
  );
}
```

usePrevious — remember last render's value:

```
// usePrevious.js - uses useRef (Part J2) to survive renders
import { useRef, useEffect } from "react";

export function usePrevious(value) {
  const ref = useRef();
  useEffect(() => { ref.current = value; }, [value]);
  return ref.current; // the value from the PREVIOUS render
}

// Usage: detect direction of change
const prevCount = usePrevious(count);
const rising = count > prevCount;
```

The habit — build your own hook library. After Phase 6 you'll have a `hooks/` folder: `useFetch`, `useLocalStorage`, `useDebounce`, `useToggle`, `usePrevious`. Copy it into every new project. Every senior React dev has this folder — start yours now. (Roadmap break-day task: "can you write each custom hook from memory?" — that's the real test.)

Part I — useReducer & Advanced State

`useState` is perfect for simple values. When state gets complex — many fields, many ways to change it — `useReducer` gives you one organised place for all the update logic. If you learned OOP's "single doorway for change" in Phase 4, this is its React form.

11. Why useReducer

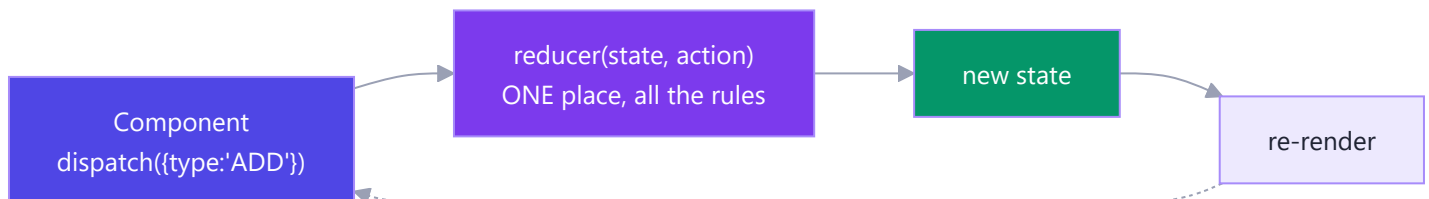
Simple definition: `useReducer` is an alternative to `useState` for **complex state** — when the next state depends on the action taken, and there are many possible actions. It centralises all update logic in one **reducer** function instead of scattering `setX` calls everywhere.

Telugu: `useReducer` ante complex state కోసం `useState` ki alternative. Eppudu? State lo **chala fields** unnadu, leda danini maaradanisi **chala vidhalu** unnadu (add, remove, update, clear...). Anni `setX` calls app anta chellacheduru undi badulu, mottu update logic ni **ok reducer function** ni pedathu. Phase 4 loni "single doorway for change" gurthundu? Reducer adu — anni maarpulu ok daari gunda.

When `useState` starts to hurt:

```
// Too many related states, too many setters - hard to keep consistent
const [items, setItems] = useState([]);
const [total, setTotal] = useState(0);
const [count, setCount] = useState(0);
const [discount, setDiscount] = useState(0);
// every add/remove has to remember to update ALL of these correctly 🤯
```

When several pieces of state change *together* and in *structured ways*, `useReducer` is cleaner: you describe *what happened* ("ADD_ITEM"), and one function computes the whole next state.



The rule of thumb: reach for `useReducer` when (a) state is an object/array with several sub-values that change together, (b) the next state depends on the previous in non-trivial ways, or (c) you have many action types. Otherwise, `useState` is simpler — don't over-engineer.

12. The reducer + dispatch Pattern

Simple definition: `useReducer(reducer, initialState)` returns `[state, dispatch]`. A **reducer** is a pure function `(state, action) => newState`. You call `dispatch(action)` to trigger a change; the reducer decides the new state based on the action's `type`.

Telugu: `const [state, dispatch] = useReducer(reducer, initial)` . రెండు bhāgālu: (1) **reducer** — `(state, action) => newState` , pure function, action **type** చూసి కొత్త state istంది. (2) **dispatch** — "idీ jరిగింది" ani reducer ki chొప్పే function: `dispatch({ type: "ADD", item })` . నంప్వ `setState` పీలవవు — *ఎంజరిగిందో* dispatch చేస్తావు, reducer *ఎం maarలో* లెక్కిస్తుంది. Action = "emi jరిగింది"; reducer = "state ela maarలీ".

The three pieces:

```
import { useReducer } from "react";

// 1. initial state
const initialState = { count: 0 };

// 2. reducer – a pure function, one switch over action types
function reducer(state, action) {
  switch (action.type) {
    case "INCREMENT": return { count: state.count + 1 };
    case "DECREMENT": return { count: state.count - 1 };
    case "SET":       return { count: action.payload };
    case "RESET":     return { count: 0 };
    default:          return state;           // unknown action → no change
  }
}

// 3. the component wires them together
function Counter() {
  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <div>
      <p>{state.count}</p>
      <button onClick={() => dispatch({ type: "INCREMENT" })}>+</button>
      <button onClick={() => dispatch({ type: "DECREMENT" })}>-</button>
      <button onClick={() => dispatch({ type: "SET", payload: 100 })}>100</button>
      <button onClick={() => dispatch({ type: "RESET" })}>Reset</button>
    </div>
  );
}
```

Read the flow: a click calls `dispatch({ type: "INCREMENT" })` . React runs `reducer(currentState, action)` , gets the new state, and re-renders. The component never computes state itself — it only *announces what happened*.

Reducers must be pure (Phase 4 J7): given the same `state` and `action` , always return the same new state; never mutate `state` , never do side effects (no `fetch` , no `Math.random`) inside. Always return a **new** object ({

`...state, ... }`), just like immutable `useState` updates (D4). This purity is exactly why reducers are easy to test and reason about.

13. `useReducer` vs `useState`

Simple definition: they solve the same problem — component state — at different complexity levels. `useState` for simple, independent values; `useReducer` for structured state with many actions. Neither is "better"; they fit different jobs.

Telugu: రెండూ ok పాని — component state — కాని *veru veru* complexity ki. `useState` = simple, *oontu* values (count, name, isOpen). `useReducer` = structured state + *chala* actions (cart, form with validation, multi-step wizard). Ok *ba* mంచిది inki *ba* chadadi kaadu — *saina chota sainadi* vadadam skill. Chalamundi modatho `useState` tho madilupethi, complex ainuvudu reducer ki maarustaru.

The comparison:

	<code>useState</code>	<code>useReducer</code>
Best for	simple, independent values	complex/structured state, many actions
Update logic	inline <code>setX(...)</code> calls	centralised in one reducer
How you change it	<code>setCount(5)</code>	<code>dispatch({ type: "SET", payload: 5 })</code>
Testability	fine	excellent (reducer is a pure function)
Boilerplate	minimal	more setup
Analogy	a light switch	a control panel with labelled buttons

The same counter, both ways:

```
// useState - perfect here, don't over-engineer
const [count, setCount] = useState(0);
setCount(count + 1);

// useReducer - overkill for one number, but shows the shape
const [state, dispatch] = useReducer(reducer, { count: 0 });
dispatch({ type: "INCREMENT" });
```

The migration signal. Start with `useState`. When you notice (a) several `setX` calls that must fire together to stay consistent, (b) the same update logic duplicated across handlers, or (c) bugs from forgetting to update one piece — that's the moment to consolidate into a reducer. Your roadmap Day 5 does exactly this: *"rewrite the Day 2 form using useReducer."*

14. Shopping Cart Example

Simple definition: the textbook `useReducer` use case — a cart with `ADD_ITEM`, `REMOVE_ITEM`, `UPDATE_QTY`, and `CLEAR_CART`. All the rules live in one reducer; components just dispatch actions. (Roadmap Q3.)

Telugu: useReducer ki classic example — shopping cart. 4 actions: `ADD_ITEM`, `REMOVE_ITEM`, `UPDATE_QTY`, `CLEAR_CART`. Cart loni anni rules (item unte qty pōchu, lēkavōthē kōtṭa add chēyi, qty 0 aḥṭē tēnēyi) okṭi reducer lī. Component kēvalaṁ `dispatch({ type: "ADD_ITEM", item })` — logic antho reducer chōṣaṣuṣuṁṭundi. Idi nī roadmap Q3.

The cart reducer — every rule in one place:

```
function cartReducer(state, action) {
  switch (action.type) {
    case "ADD_ITEM": {
      const existing = state.find(i => i.id === action.item.id);
      if (existing) {
        // already in cart → bump quantity (immutable map, D4)
        return state.map(i =>
          i.id === action.item.id ? { ...i, qty: i.qty + 1 } : i
        );
      }
      return [...state, { ...action.item, qty: 1 }]; // new item
    }
    case "REMOVE_ITEM":
      return state.filter(i => i.id !== action.id);

    case "UPDATE_QTY":
      return state
        .map(i => i.id === action.id ? { ...i, qty: action.qty } : i)
        .filter(i => i.qty > 0); // qty 0 → drop it

    case "CLEAR_CART":
      return [];

    default:
      return state;
  }
}
```

The component — pure dispatch, zero logic:

```
function Cart() {
  const [cart, dispatch] = useReducer(cartReducer, []);

  const total = cart.reduce((sum, i) => sum + i.price * i.qty, 0); // Phase 4 reduce!

  return (
    <div>
      {cart.map(item => (
        <div key={item.id}>
          {item.name} × {item.qty}
          <button onClick={() => dispatch({ type: "UPDATE_QTY", id: item.id, qty: item.qty + 1 })}>
          <button onClick={() => dispatch({ type: "REMOVE_ITEM", id: item.id })}>Remove</button>
        </div>
      ))}
      <p>Total: ₹{total}</p>
      <button onClick={() => dispatch({ type: "CLEAR_CART" })}>Clear</button>
    </div>
  );
}
```

Notice how every array update inside the reducer is the **immutable** pattern from D4 (`map` , `filter` , spread) and the total uses Phase 4's `reduce` . `useReducer` isn't new magic — it's your existing array skills, organised behind labelled actions.

15. useReducer + Context = Mini-Redux

Simple definition: put a reducer's `state` and `dispatch` into a **Context**, and any component in the tree can read the state or dispatch actions — no prop drilling. This is the core idea behind Redux, hand-built with two hooks.

Telugu: Reducer యొక్క `state` mariyu `dispatch` ni **Context** లో **పెడితే** — app anta ఏ component నైనా state చదవొచ్చు, action dispatch చేయొచ్చు, props లేకుండా. Idరే **Redux** venuka unna core idea — కాని నంవ్వు కేవలం 2 hooks (useReducer + useContext) tho కట్టేస్తున్నావు. Peddha apps lo global state (cart, auth, notifications) ki idరీ వునాది.

Combine the two patterns:

```
// CartContext.jsx
import { createContext, useContext, useReducer } from "react";

const CartContext = createContext();

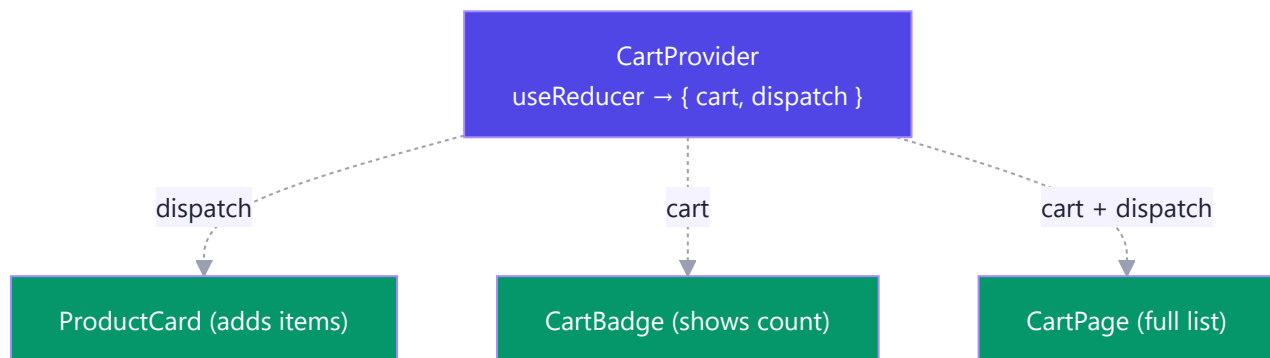
export function CartProvider({ children }) {
  const [cart, dispatch] = useReducer(cartReducer, []);
  return (
    <CartContext.Provider value={{ cart, dispatch }}>
      {children}
    </CartContext.Provider>
  );
}

export const useCart = () => useContext(CartContext);
```

Now any component, anywhere, controls the cart:

```
// A product card deep in the tree adds to the cart – no props threaded
function ProductCard({ product }) {
  const { dispatch } = useCart();
  return (
    <button onClick={() => dispatch({ type: "ADD_ITEM", item: product })}>
      Add to cart
    </button>
  );
}

// The navbar badge reads the same cart
function CartBadge() {
  const { cart } = useCart();
  return <span>🛒 {cart.length}</span>;
}
```



Why this matters: you just built the essence of Redux/Zustand with built-in React. For most apps, `useReducer` + Context is *enough* — you often don't need a state-management library at all. When you later meet Redux Toolkit, you'll recognise it as "this pattern, with dev-tools and conventions added." (SAP UI5's models play a similar centralised-state role — the concept keeps transferring.)

Part J — Performance & the Rest of the Hooks

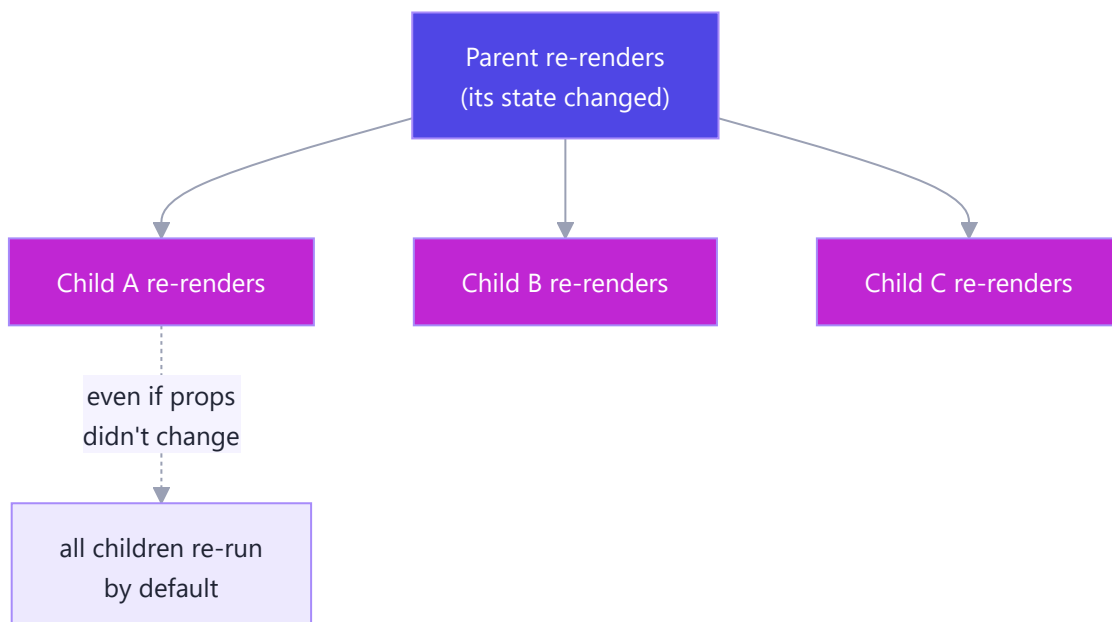
The core hooks (`useState`, `useEffect`, `useReducer`, `useContext`) build every app. This part covers the performance hooks (`useMemo`, `useCallback`, `React.memo`) and `useRef` — the tools you add when an app grows. Learn what they do; use them only when you measure a real need.

J1. The Re-Render Mental Model

Simple definition: a component re-renders when its **state changes**, its **props change**, or its **parent re-renders**. "Re-render" means React calls the function again to compute new JSX — it does *not* mean the whole DOM is rebuilt.

Telugu: Component eppudu malli render avuthundi? (1) daani **state maarithe**, (2) daani **props maarithe**, (3) daani **parent re-render aithe**. "Re-render" ante React aa function ni malli pethuchi kottu JSX lekunchadam — mottu DOM malli kuthadam kuda (React kavalu maarina bhaganni matrame update chesundi). Idi teliste, performance hooks **enduku** unnayoi artham avuthundi.

The key insight — parents re-render children by default. When a parent re-renders, *all* its children re-render too, even if their props didn't change. Usually this is cheap and fine — React is fast. But in large trees or expensive components, needless re-renders add up.



The golden rule of optimisation: don't — until you measure. React is fast enough that most apps never need these hooks. Premature `useMemo` / `useCallback` everywhere adds complexity for no gain. Use React DevTools'

Profiler to *find* a slow render first, then optimise that spot. The tools below are for when you have a real, measured problem — which is why your roadmap marks them "read-only today."

J2. useRef — the Box That Survives Renders

Simple definition: `useRef` gives you a mutable box (`ref.current`) that **persists across renders but does not trigger a re-render** when changed. Two main uses: referencing a DOM element, and remembering a value without causing renders.

Telugu: `useRef` ఒక **dabbā** (`ref.current`) — renders madhya value ni luస్తుంది, kani maari na **re-render** kardu (useState ki idr tirda!). Rendu vadakalu: (1) **DOM element ni pottukovadam** — input ki focus potladu laantivi. (2) render trigger cheyusunda **value gathupettukovadam** — timer id, previous value. State kardu kani niliche jhapaku.

Use 1 — reference a DOM element (focus an input):

```
import { useRef } from "react";

function SearchBar() {
  const inputRef = useRef(null);

  useEffect(() => {
    inputRef.current.focus(); // focus the input on mount
  }, []);

  return <input ref={inputRef} placeholder="Search..." />;
}
```

Use 2 — remember a value without re-rendering (a timer id):

```
function Stopwatch() {
  const [time, setTime] = useState(0);
  const intervalRef = useRef(null); // survives renders, no re-render

  const start = () => {
    intervalRef.current = setInterval(() => setTime(t => t + 1), 1000);
  };
  const stop = () => clearInterval(intervalRef.current);
  // ...
}
```

`useRef` vs `useState` — the crucial difference:

	<code>useState</code>	<code>useRef</code>
Survives re-renders	❌	✅

	useState	useRef
Changing it re-renders	✓ yes	✗ no
Use for	data shown on screen	DOM refs, timer ids, "instance variables"

The rule: if changing the value should update the screen, use `useState`. If it's behind-the-scenes bookkeeping (a DOM node, an interval id, a "did this already run" flag), use `useRef`. Putting a timer id in state would re-render pointlessly; putting displayed data in a ref would fail to update the screen.

J3. useMemo

Simple definition: `useMemo(fn, deps)` **caches the result** of an expensive calculation and only re-computes it when a dependency changes. It skips redundant work on re-renders.

Telugu: `useMemo` ఒక ఖరీదైన లెక్క, యొక్క result ని **దాచుకుంటుంది** (cache), `deps` మారినప్పుడే `malli` లెక్కిస్తుంది.

Example: 10,000 items ni sort/filter చేయడం prati render ki avసరం లేదు — data మారినప్పుడే చాలు.

`useMemo(() => heavyCalc(items), [items])`. Warning: చేసిన లెక్కలకి వాడకు — cache maintain చేయడం కూడా cost, చేసిన దానికీ waste.

Caching an expensive computation:

```
function ProductList({ products, filter }) {
  // Without useMemo, this sort re-runs on EVERY render, even unrelated ones
  const visible = useMemo(() => {
    return products
      .filter(p => p.name.includes(filter))
      .sort((a, b) => a.price - b.price); // pretend this is expensive
  }, [products, filter]); // recompute only when these change

  return visible.map(p => <ProductCard key={p.id} {...p} />);
}
```

The other job of `useMemo` — stable object/array references. Recall the E5 gotcha: a new object every render breaks dependency arrays and `React.memo`. `useMemo` fixes it by keeping the *same* reference until `deps` change:

```
const options = useMemo(() => ({ limit, sort }), [limit, sort]);
// now `options` is stable - safe to use in a dependency array
```

When to use it (measure first): genuinely expensive calculations (sorting/filtering thousands of items, heavy math) run on every render, or when you need a stable reference to satisfy `React.memo` / `deps`. For a `.map` over 20 items or `a + b`, skip it — the memoization overhead costs more than the calculation.

J4. useCallback

Simple definition: `useCallback(fn, deps)` is `useMemo` for **functions** — it returns the *same function reference* across renders (until deps change). Its main use is passing stable callbacks to memoized child components.

Telugu: `useCallback` ante functions కోసం `useMemo`. Prati render lo functions **కొత్తగా** పడతాయి (కొత్త reference). Aa కొత్త function ni memoized child ki prop ga pampithe, child "prop maarindi!" ani anukuni malli render avuntundi — vudda. `useCallback` aa function ni **stable** ga unchuntundi. `useMemo(value)`, `useCallback(function)` — oki idea, veru target.

The problem it solves — new function every render:

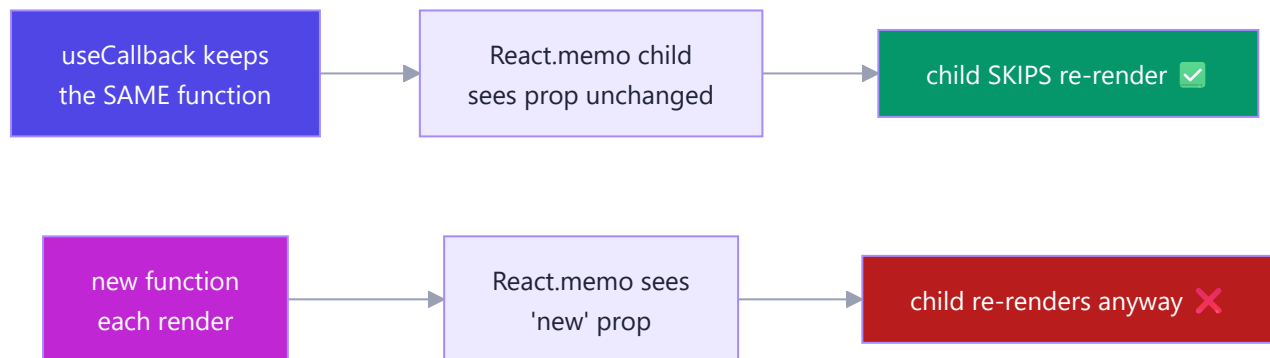
```
function Parent() {
  const [count, setCount] = useState(0);

  // ❌ a BRAND NEW function every render → breaks React.memo on the child
  const handleClick = () => console.log("clicked");

  // ✅ same function reference across renders
  const handleClick2 = useCallback(() => console.log("clicked"), []);

  return <ExpensiveChild onClick={handleClick2} />;
}
```

Why it matters only with `React.memo` . Passing a fresh function to a *normal* child is harmless (it was going to re-render anyway). `useCallback` earns its keep when the child is wrapped in `React.memo` (next section) — then a stable function prop lets the child actually skip re-rendering.



The rule: `useCallback` and `React.memo` are a **team** — using one without the other is usually pointless. Don't wrap every handler in `useCallback` reflexively; do it when you're passing a callback to a memoized child and have measured re-renders as a problem.

J5. React.memo

Simple definition: `React.memo(Component)` wraps a component so it **skips re-rendering when its props haven't changed**. It's the tool that stops "parent re-rendered → all children re-render" for expensive children.

Telugu: `React.memo(Component)` ఒక component ni చూపిస్తే, దాని **props maarకపోతే re-render skip** చేస్తుంది. J1 lo "parent re-render అయితే anni children re-render" ani చెప్పాం kada — expensive child ki adhi vaddunte `React.memo` వాడు. Idi `useCallback` / `useMemo` tho jath kalinestene panichevuntundi — Ithavath props prati saari kottuna kanipinchi skip jaraduni.

Wrapping a component to skip needless re-renders:

```
import { memo } from "react";

// This child only re-renders when `product` actually changes,
// not every time the parent re-renders for unrelated reasons.
const ProductCard = memo(function ProductCard({ product, onAdd }) {
  console.log("rendering", product.name);
  return (
    <div>
      {product.name}
      <button onClick={() => onAdd(product.id)}>Add</button>
    </div>
  );
});
```

For `memo` to actually skip, **every prop must be stable**: primitives (strings, numbers) compare fine, but objects/arrays/functions need `useMemo` / `useCallback` in the parent, or `memo` sees a "new" prop every time and re-renders anyway.

The performance trio — how they fit together:

Hook/API	Memoizes	Pair it with
<code>React.memo</code>	a component (skip render if props equal)	stable props below
<code>useMemo</code>	a value (skip recompute)	<code>React.memo</code> children, deps
<code>useCallback</code>	a function (stable reference)	<code>React.memo</code> children

The honest summary: these three are for measured performance problems in larger apps. A typical Phase 6 / Task Tracker app **does not need them** — and adding them everywhere makes code harder to read for zero benefit. Know what they do (interviews ask), recognise them in codebases, and apply them surgically when the Profiler points at a real slow spot. Correctness and clarity first; optimise second.

J6. The Hooks Rulebook (Recap)

Simple definition: a one-screen reference to every hook you've met, what it's for, and the two rules that govern all of them.

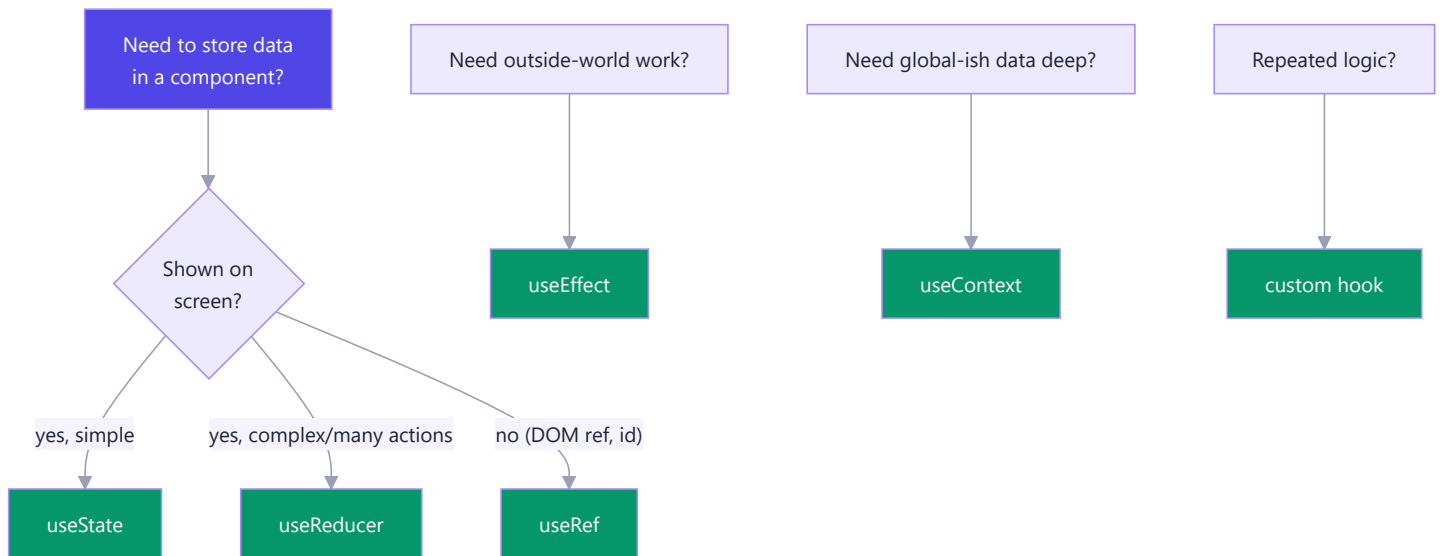
Telugu: నాంవ్వు నీర్చుకున్న **anni hooks** ok్ చota — ఏది deniki, ఎప్పుడు వాడాలి. Rెండు rules mళ్ళీ (H1): (1) hooks ni ఎప్పుడూ **top-level** లో matrame పీలువు (if/loop lopala vద్దు). (2) hooks ni **components లేదా custom hooks** నాంచే పీలువు. Ee table ni గూర్చుపెట్టుకుంటే, "ee paniki ఏ hook?" ane prశ్న ki jవాబు వేంటనే వస్తుంది.

Every hook at a glance:

Hook	What it does	Reach for it when
<code>useState</code>	local component state	any value that changes over time
<code>useEffect</code>	run side effects after render	fetch, subscribe, sync to outside world
<code>useContext</code>	read a Context value	avoid prop drilling of global-ish data
<code>useReducer</code>	complex state via a reducer	many actions / structured state
<code>useRef</code>	mutable box, no re-render	DOM refs, timer ids, instance vars
<code>useMemo</code>	cache a computed value	expensive calc / stable reference
<code>useCallback</code>	cache a function reference	stable callback to a <code>memo</code> child
custom <code>useXxx</code>	reuse stateful logic	any repeated pattern (Part H)

The two Rules of Hooks (never break):

- Top level only** — not inside `if`, loops, or nested functions. React tracks hooks by call order.
- React functions only** — components or custom hooks, never plain functions or event handlers.



The keyboard test (Phase 4 L8, React edition): you *know* a hook when you can (1) explain what it does, (2) write it from a blank file, and (3) debug it when it misbehaves at 11pm. Reading this table gets you to level 1 — building the Task Tracker (Part L) gets you to 3.

Part K — Tailwind CSS

React builds the structure; Tailwind makes it beautiful — fast. Instead of writing CSS in separate files, you style directly in your JSX with tiny utility classes. It feels wrong for five minutes, then you never want to go back.

K1. What is Tailwind & Why Utility-First

Simple definition: Tailwind is a **utility-first** CSS framework. Instead of writing custom CSS rules, you apply many small, single-purpose classes (`p-4` , `text-center` , `bg-blue-500`) directly in your markup. Each class does exactly one thing.

Telugu: Tailwind ఒక **utility-first** CSS framework. Separate CSS file lo rules raayukunda, chinnu chinnu single-purpose classes ni **నేరుగా JSX lo** pedita: `p-4` (padding), `text-center` (center text), `bg-blue-500` (blue background). Prati class oki paani chistundi. Modata vintana anipistundi ("markup lo enni classes!"), kani 5 nimishalu taruvata chala **fast** ga UI kisthuvu — CSS file ki, HTML ki madhya jampin undu.

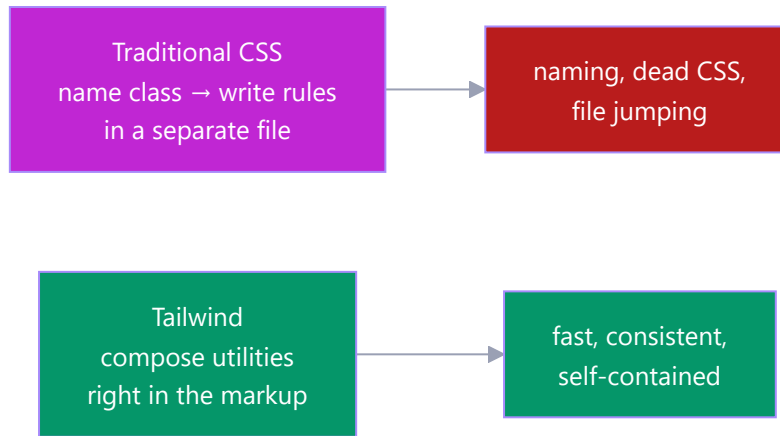
Traditional CSS vs Tailwind — the same button:

```
// Traditional: name a class, then define it elsewhere in a .css file
<button className="btn-primary">Save</button>
/* styles.css */
.btn-primary { padding: 8px 16px; background: #3b82f6; color: white; border-radius: 6px; }

// Tailwind: the styles ARE the classes — no separate file, no naming
<button className="px-4 py-2 bg-blue-500 text-white rounded-md">Save</button>
```

Why utility-first wins (once it clicks):

- **No naming things** — you don't invent `.btn-primary` , `.card-header-wrapper` . Naming is one of the hard problems in CSS; Tailwind deletes it.
- **No context switching** — style right where you build, no jumping to a `.css` file.
- **No dead CSS** — you never wonder "is this class still used?" Delete the element, the styles go with it.
- **Consistent by design** — `p-4` , `p-6` , `p-8` come from a fixed scale, so spacing/colors stay uniform across the app.
- **Responsive & dark mode inline** — `md:` , `dark:` prefixes (K3, K4) without media-query files.



Setup (Vite): install Tailwind, add the plugin, add one `@import "tailwindcss";` line to your CSS. (The docs' "Framework Guides → Vite" is the exact recipe; it's a 3-minute one-time setup.) Once done, every Tailwind class just works in your JSX.

The common objection — "the markup looks ugly with 10 classes!" True at first. But those classes are *co-located* with the element, they're consistent, and when you have repetition you extract a **component** (K5) — not a CSS class. The React component *is* your reusable unit, so Tailwind and React fit together perfectly.

K2. The Core Utilities

Simple definition: ~90% of styling uses a small vocabulary of utilities — spacing, sizing, color, typography, and layout (flex/grid). Learn these families and you can build almost anything.

Telugu: Tailwind lo v^l classes unn^o, **rojav^o pa^l 90%** iv^e k^onni families: **spacing** (padding/margin), **size** (width/height), **color** (bg/text), **typography** (font size/weight), **layout** (flex/grid). Ee groups artham a^hth^e, migⁱlⁱnvⁱ docs lo ch^ova^osu^oku^ont^ova^o ra^omo^ochu. Number scale ga^ort^otp^ol^oko: **4** = 1rem = 16px (so **p-4** = 16px padding).

Spacing — padding & margin (**p** / **m** + side + scale):

```
<div className="p-4">      {/* padding all sides: 16px */}
<div className="px-6 py-2"> {/* padding x (left+right) 24px, y 8px */}
<div className="mt-4 mb-2"> {/* margin-top 16px, margin-bottom 8px */}
<div className="m-auto">    {/* auto margins → center a block */}
```

The scale: **1** = 4px, **2** = 8px, **4** = 16px, **6** = 24px, **8** = 32px... (each step = 4px). Sides: **t** top, **b** bottom, **l** left, **r** right, **x** horizontal, **y** vertical.

Sizing — width & height:

```
<div className="w-full">      {/* width: 100% */}
<div className="w-64">        {/* width: 16rem (256px) */}
<div className="h-screen">    {/* height: 100vh */}
<div className="max-w-md mx-auto">  {/* readable column, centered */}
```

Color — background, text, border:

```
<div className="bg-blue-500 text-white">    {/* Tailwind's color-shade scale */}
<p    className="text-slate-600">          {/* 50 (lightest) → 950 (darkest) */}
<div className="border border-gray-200">
```

Colors follow `color-shade`: `blue-500`, `slate-600`, `red-100`. Shades go `50, 100, 200 ... 900, 950`. `500` is the "base" of each color.

Typography:

```
<h1 className="text-3xl font-bold">          {/* size + weight */}
<p  className="text-sm text-gray-500 leading-relaxed">
<span className="uppercase tracking-wide">    {/* letter-spacing */}
```

Sizes: `text-xs, sm, base, lg, xl, 2xl, 3xl...`. Weights: `font-light, normal, medium, semibold, bold`.

Borders & effects:

```
<div className="rounded-lg shadow-md border border-gray-100">
<button className="rounded-full shadow-lg">
```

Flexbox — the everyday layout tool:

```
<div className="flex items-center justify-between gap-4">
  {/* flex row · vertical-center · space-between · 16px gap */}
  <Logo />
  <Nav />
</div>

<div className="flex flex-col gap-2">    {/* vertical stack */}
```

Key flex utilities: `flex` (row), `flex-col` (column), `items-center` (cross-axis center), `justify-between` / `justify-center` (main-axis), `gap-4` (spacing between children).

Grid — for card layouts:

```
<div className="grid grid-cols-3 gap-4">    {/* 3 equal columns */}
```

Family	Example classes	Does
Spacing	<code>p-4</code> <code>px-6</code> <code>mt-2</code> <code>gap-4</code>	padding, margin, gaps
Sizing	<code>w-full</code> <code>h-screen</code> <code>max-w-md</code>	width, height
Color	<code>bg-blue-500</code> <code>text-white</code>	background, text color
Typography	<code>text-xl</code> <code>font-bold</code> <code>text-center</code>	size, weight, align
Border/FX	<code>rounded-lg</code> <code>shadow-md</code> <code>border</code>	corners, shadow, borders

Family	Example classes	Does
Flex	<code>flex</code> <code>items-center</code> <code>justify-between</code>	flexbox layout
Grid	<code>grid</code> <code>grid-cols-3</code>	grid layout

K3. Responsive Design

Simple definition: Tailwind is **mobile-first**. A plain class applies everywhere; adding a breakpoint prefix (`sm:` , `md:` , `lg:`) makes it apply *only at that screen width and up*. You build responsive layouts without writing a single media query.

Telugu: Tailwind **mobile-first** — prefix లేని class anni screens ki. Prefix (`md:` , `lg:`) పడితే aa **size మరియు అపైన** matrame apply అవుతుంది. So `grid-cols-1 md:grid-cols-3` ante: "చిన్న screen lo 1 column, medium+ lo 3 columns". Media queries okక్కటి రాయకుండా responsive! Rule: **చిన్న screen ki modata raayi** (prefix లేకుండా), అపైన పడ్డ screens ki prefix tho over-ride చేయి.

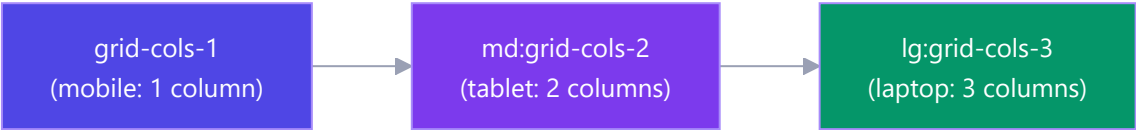
The breakpoints:

Prefix	Applies at width ≥	Typical device
(none)	0px (always)	mobile (base)
<code>sm:</code>	640px	large phone
<code>md:</code>	768px	tablet
<code>lg:</code>	1024px	laptop
<code>xl:</code>	1280px	desktop

A responsive card grid — the roadmap's Day 5 exercise:

```
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
  {tasks.map(t => (
    <article key={t.id} className="p-4 rounded-lg bg-white shadow">
      <h3 className="font-semibold">{t.title}</h3>
    </article>
  ))}
</div>
```

Read the grid classes: **1 column on mobile, 2 on tablets** (`md:`), **3 on laptops** (`lg:`). Resize the browser and watch it reflow — no media queries anywhere.



Responsive everything, not just grids:

```
<div className="flex flex-col md:flex-row">    {/* stack on mobile, row on desktop */}
<h1 className="text-2xl md:text-4xl">        {/* bigger heading on bigger screens */}
<div className="hidden md:block">            {/* hide on mobile, show on desktop */}
<div className="p-4 md:p-8">                {/* more breathing room on desktop */}
```

The mobile-first mindset: write the phone layout with unprefix classes, then *add* prefixes to enhance for larger screens. This is why `grid-cols-1` has no prefix and `md:grid-cols-3` does — you start small and scale up, which matches how most of your users actually browse (phones).

K4. Hover, Focus & Dark Mode

Simple definition: state variants are prefixes that apply a utility only in a certain interaction state — `hover:`, `focus:`, `active:`, `disabled:` — and `dark:` for dark mode. Same idea as breakpoints, applied to states instead of screen sizes.

Telugu: Breakpoints లాగే, **state variants** కూడా prefixes — కానీ screen size కంటే, **interaction state** batti apply avantay. `hover:bg-blue-600` = mouse paina unappadu matrame. `focus:ring-2` = input click chesina vapudu. `disabled:opacity-50` = button off unappadu. `dark:bg-slate-900` = dark mode lo matrame. Ee prefixes tho interactive, dark-mode-ready UI — extra CSS lathundi.

Interaction states — hover, focus, active, disabled:

```
<button className="
  bg-blue-500 text-white px-4 py-2 rounded
  hover:bg-blue-600          /* darker on mouse-over */
  active:bg-blue-700         /* even darker while clicking */
  focus:ring-2 focus:ring-blue-300 /* focus ring for keyboard users */
  disabled:opacity-50 disabled:cursor-not-allowed
">
  Save
</button>
```

Dark mode — the `dark:` variant:

```
<div className="
  bg-white text-slate-900      /* light mode */
  dark:bg-slate-900 dark:text-white /* dark mode */
">
  <h1 className="text-slate-800 dark:text-slate-100">Dashboard</h1>
</div>
```

Connecting dark mode to your ThemeContext (G3). Tailwind's `dark:` classes activate when a `dark` class is on a parent (usually `<html>`). So your `ThemeContext` toggle just adds/removes that class:

```
// In your ThemeProvider's effect:
useEffect(() => {
  document.documentElement.classList.toggle("dark", theme === "dark");
}, [theme]);
```

Now the `useTheme()` toggle button from Part G3 flips *every* `dark:` class in the app at once. Context holds the *state*; Tailwind's `dark:` does the *styling*. Two Phase-6 tools clicking together.



You can stack prefixes: `md: hover: bg-blue-600` (hover effect, but only on tablet+), `dark: hover: bg-slate-700` (hover color in dark mode). Order reads left-to-right as "at this breakpoint, in this state."

K5. Reusing Styles — Components, `@apply`, `clsx`

Simple definition: when the same class list repeats, don't copy-paste it. In React the primary answer is to **extract a component**. For the rare cases where you want a reusable CSS class, `@apply` bundles utilities; for conditional classes, `clsx` / `cn` keeps them clean.

Telugu: Okkē pōḍavaina class list mēllī mēllī vāṇṇē, copy-paste chēyuku. React lo **modati answer: component ga extract chēyali** (Button, Card). Aa component okkē chota styles, everywhere vōḍōchupu — Tailwind + React yōṇṇē natural fit. Okvēḷa CSS class kavalaṇṇē `@apply`; conditional classes ki `clsx`. Kōṇi 90% saarḷo — **component ye correct answer**.

The React way — extract a component (do this 90% of the time):

```
// Instead of pasting these 6 classes on every button...
function Button({ children, variant = "primary", ...props }) {
  const base = "px-4 py-2 rounded-md font-medium transition";
  const variants = {
    primary: "bg-blue-500 text-white hover:bg-blue-600",
    danger: "bg-red-500 text-white hover:bg-red-600",
    ghost: "bg-transparent text-slate-700 hover:bg-slate-100",
  };
  return (
    <button className={`${base} ${variants[variant]}`} {...props}>
      {children}
    </button>
  );
}

// Now every button is consistent AND short:
<Button>Save</Button>
<Button variant="danger">Delete</Button>
```

The repeated styles live in *one component*, configured by props — this is Part C's reusable `<Button>`, now the answer to "how do I not repeat Tailwind classes?"

`@apply` — **bundle utilities into a CSS class** (use sparingly, for truly global primitives):

```
/* index.css */
.btn {
  @apply px-4 py-2 rounded-md font-medium;
}
```

```
<button className="btn bg-blue-500 text-white">Save</button>
```

Reach for `@apply` only for app-wide primitives (`.btn` , `.input`) — over-using it recreates the naming problem Tailwind removed.

`clsx` / `cn` — **clean conditional classes**:

```
import clsx from "clsx";    // npm install clsx

<button
  className={clsx(
    "px-4 py-2 rounded",      // always
    isActive && "bg-blue-500 text-white",  // when active
    isDisabled && "opacity-50 cursor-not-allowed",
  )}
>
```

`clsx` joins classes and drops the falsy ones — far cleaner than nested template-string ternaries when you have several conditional styles. Without it, conditional Tailwind turns into an unreadable ``px-4 ${a ? "x" : ""} ${b ? "y" : ""}``.

The hierarchy of reuse: (1) extract a **component** → (2) use `clsx` for conditional variants → (3) `@apply` only for a few global primitives. In that order.

K6. Tailwind + React Patterns

Simple definition: the everyday patterns for using Tailwind *inside* React — dynamic classes from props/state, style maps, and combining conditional logic with the class string. This is where the two technologies actually meet.

Telugu: Tailwind ni React lopala rojvārī ela vāḍāḷō — props/state batti **dynamic classes, style maps** (object lo variants), mriyū conditional logic ni class string tho klapaḍo. Idī Phase 6 yōḱṣa, "React + Tailwind" — rēṇḍa kalīṇi panīcēṇē chōṭi. Peddha pattern: **object lookup** — variant ni key ga vāḍi, class string ni tīyāḍo (Badge example gurthundō?).

Pattern 1 — dynamic classes with template literals:

```
function Alert({ type, children }) {
  return (
    <div className={`p-4 rounded-lg border ${
      type === "error" ? "bg-red-50 border-red-200 text-red-700"
      : "bg-green-50 border-green-200 text-green-700"
    }`} >
      {children}
    </div>
  );
}
```

Pattern 2 — the style map (cleanest for 3+ variants):

```
function Badge({ text, tone = "info" }) {
  const tones = {
    info: "bg-blue-100 text-blue-700",
    success: "bg-green-100 text-green-700",
    warn: "bg-amber-100 text-amber-700",
    error: "bg-red-100 text-red-700",
  };
  return <span className={`px-2 py-1 rounded text-sm ${tones[tone]}`}>{text}</span>;
}
```

This is the *exact* **Badge** from your roadmap's Day 1 snippet — object lookup (`tones[tone]` , Phase 4 D3) picks the class string. It's the professional way to do variants: no ternary chains, easy to add a new tone.

Pattern 3 — conditional classes with state:

```
function Tab({ label, active, onClick }) {
  return (
    <button
      onClick={onClick}
      className={clsx(
        "px-4 py-2 font-medium border-b-2",
        active ? "border-blue-500 text-blue-600" : "border-transparent text-slate-500"
      )}
    >
      {label}
    </button>
  );
}
```

Pattern 4 — a real card component tying it together:

```
function TaskCard({ task, onToggle }) {
  return (
    <article className="p-4 rounded-lg bg-white shadow hover:shadow-md transition dark:bg-slate-800">
      <div className="flex items-center justify-between gap-3">
        <h3 className={clsx(
          "font-semibold",
          task.done && "line-through text-slate-400" // strike done tasks
        )}>
          {task.title}
        </h3>
        <input type="checkbox" checked={task.done} onChange={() => onToggle(task.id)} />
      </div>
    </article>
  );
}
```

The mental model for React + Tailwind: React decides *what* to show (state, props, `.map`, conditionals); Tailwind decides *how it looks* (utility classes, `dark:`, `md:`, `hover:`); and when styles repeat, a **component** captures both. Structure from React, style from Tailwind, reuse from components — that's the entire Phase 6 toolkit working as one.

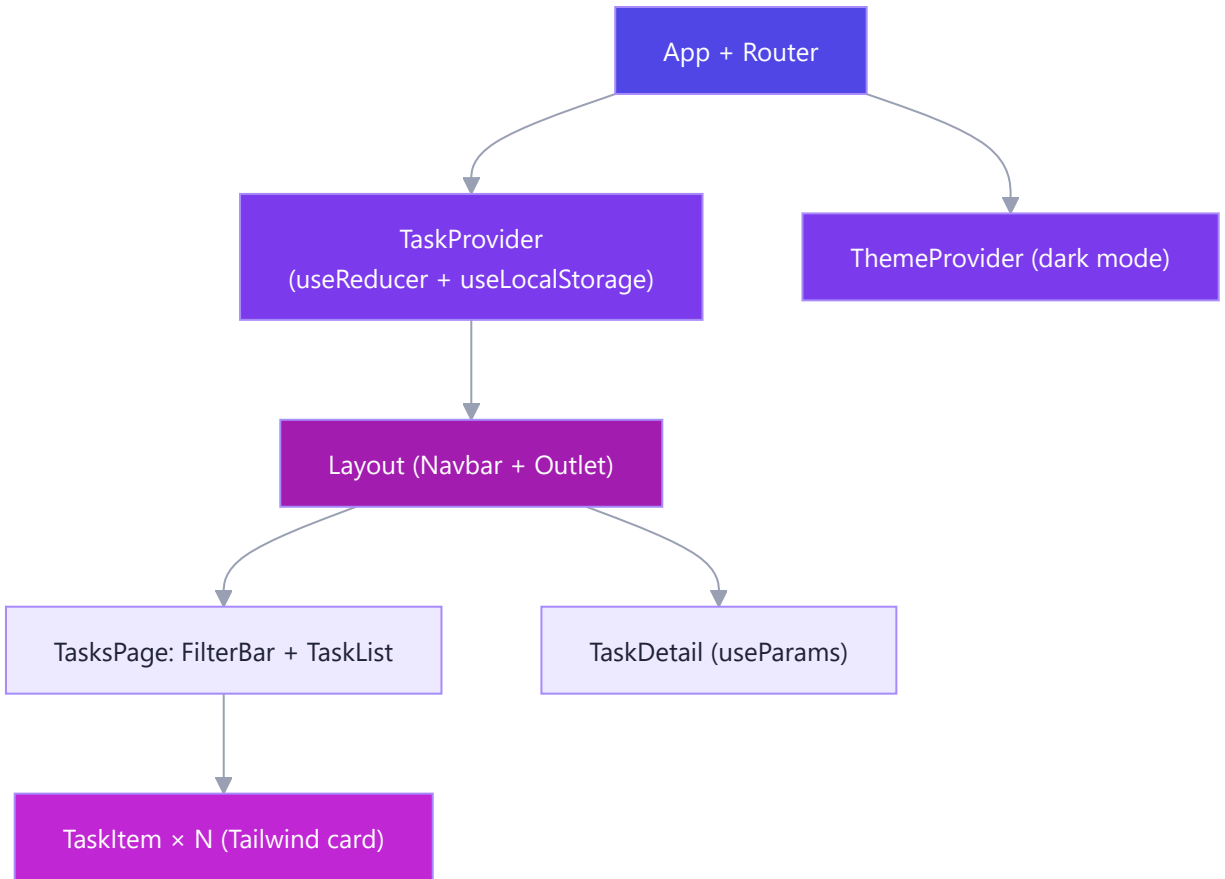
Part L — Master Program: The Task Tracker

Your Phase 6 capstone (roadmap Day 6). Every concept from Parts A–K, assembled into one real, deployable app. Read it top to bottom — the comments name the concept each line demonstrates. Then build it yourself from a blank folder; that's the deliverable.

What it does: add / edit / delete tasks · filter by All / Active / Done · persist to localStorage · light/dark theme · routing to a detail page. **The stack:** `useReducer` (task logic) + Context (share it) + custom hooks (`useLocalStorage`) + Router (pages) + Tailwind (styling).

Telugu: Idీ nī Phase 6 **capstone** — anni concepts okī app lo. Chadvadam okā ettu, **blank folder nānci nāvēśya kṛtadam** asatnē test (roadmap Day 6 deliverable). Prati bhāgaṁnī okkṛtīga chādu — reducer (task logic), Context (share), custom hook (persist), Router (pages), Tailwind (look). Ivi separate ga nīrṣṭakunnāvu; ippādu **okī chota kṛtisi** eḷa panīcēsṭayō chādu.

The architecture:



Step 1 — the reducer (all task rules in one place, Part I):

```

// taskReducer.js
export function taskReducer(state, action) {
  switch (action.type) {
    case "ADD":
      return [...state, {                                // immutable add (D4)
        id: crypto.randomUUID(),                        // stable id, never index (B5)
        title: action.title,
        done: false,
      }];
    case "TOGGLE":                                     // the famous React list update
      return state.map(t => t.id === action.id ? { ...t, done: !t.done } : t);
    case "EDIT":
      return state.map(t => t.id === action.id ? { ...t, title: action.title } : t);
    case "DELETE":
      return state.filter(t => t.id !== action.id);
    case "CLEAR_DONE":
      return state.filter(t => !t.done);
    default:
      return state;
  }
}

```

Step 2 — Context + useReducer + persistence (Parts G, H, I combined):

```
// TaskContext.jsx
import { createContext, useContext, useReducer, useEffect } from "react";
import { taskReducer } from "../taskReducer";
import { useLocalStorage } from "../hooks/useLocalStorage";

const TaskContext = createContext();

export function TaskProvider({ children }) {
  // seed the reducer from localStorage; a lazy initializer reads it once
  const [saved, setSaved] = useLocalStorage("tasks", []);
  const [tasks, dispatch] = useReducer(taskReducer, saved);

  useEffect(() => { setSaved(tasks); }, [tasks]); // persist on every change

  return (
    <TaskContext.Provider value={{ tasks, dispatch }}>
      {children}
    </TaskContext.Provider>
  );
}

export const useTasks = () => useContext(TaskContext);
```

Step 3 — the routed app shell (Part F):

```
// App.jsx
import { BrowserRouter, Routes, Route } from "react-router-dom";

function App() {
  return (
    <ThemeProvider>
      <TaskProvider>
        <BrowserRouter>
          <Routes>
            <Route element={<Layout />}>          {/* navbar + Outlet (F6) */}
              <Route path="/" element={<TasksPage />} />
              <Route path="/tasks/:id" element={<TaskDetail />} />
              <Route path="*" element={<NotFound />} />
            </Route>
          </Routes>
        </BrowserRouter>
      </TaskProvider>
    </ThemeProvider>
  );
}
```

Step 4 — the main page: add form + filter + list (Parts B, C, D):

```
// TasksPage.jsx
import { useState } from "react";
import { useTasks } from "../TaskContext";

function TasksPage() {
  const { tasks, dispatch } = useTasks();
  const [title, setTitle] = useState(""); // controlled input (D6)
  const [filter, setFilter] = useState("all"); // all | active | done

  // narrow → the six shapes from Phase 4, now feeding JSX
  const visible = tasks.filter(t =>
    filter === "active" ? !t.done :
    filter === "done" ? t.done : true
  );

  function handleAdd(e) {
    e.preventDefault(); // stop reload (E/D6)
    if (!title.trim()) return; // guard clause
    dispatch({ type: "ADD", title: title.trim() });
    setTitle(""); // clear the input
  }

  return (
    <div className="max-w-lg mx-auto p-4">
      <form onSubmit={handleAdd} className="flex gap-2 mb-4">
        <input
          value={title}
          onChange={e => setTitle(e.target.value)}
          placeholder="What next?"
          className="flex-1 px-3 py-2 border rounded-lg dark:bg-slate-800"
        />
        <button className="px-4 py-2 bg-blue-500 text-white rounded-lg hover:bg-blue-600">
          Add
        </button>
      </form>

      <FilterBar filter={filter} setFilter={setFilter} />

      <ul className="space-y-2 mt-4">
        {visible.map(task => ( // /* .map + key (B5) */
          <TaskItem key={task.id} task={task} />
        ))}
      </ul>
    </div>
  );
}
```

```

    {visible.length === 0 && (
      /* conditional (B4) */
      <p className="text-center text-slate-400 mt-6">No tasks here 🎉</p>
    )}
  </div>
);
}

```

Step 5 — the FilterBar (props + Tailwind conditional classes, Parts C, K):

```

function FilterBar({ filter, setFilter }) {
  const options = ["all", "active", "done"];
  return (
    <div className="flex gap-2">
      {options.map(opt => (
        <button
          key={opt}
          onClick={() => setFilter(opt)}
          className={`px-3 py-1 rounded-full text-sm capitalize ${
            filter === opt ? "bg-blue-500 text-white" : "bg-slate-100 text-slate-600"
          }`}
        >
          {opt}
        </button>
      ))}
    </div>
  );
}

```

Step 6 — the TaskItem card (dispatch + Link + Tailwind, Parts F, I, K):

```

import { Link } from "react-router-dom";

function TaskItem({ task }) {
  const { dispatch } = useTasks();
  return (
    <li className="flex items-center gap-3 p-3 rounded-lg bg-white shadow-sm dark:bg-slate-800">
      <input
        type="checkbox"
        checked={task.done}
        onChange={() => dispatch({ type: "TOGGLE", id: task.id })}
      />
      <Link
        to={` /tasks/${task.id}`}
        className={`flex-1 ${task.done ? "line-through text-slate-400" : ""}`}
      >
        {task.title}
      </Link>
      <button
        onClick={() => dispatch({ type: "DELETE", id: task.id })}
        className="text-red-500 hover:text-red-700"
      >
        ×
      </button>
    </li>
  );
}

```

Step 7 — the detail page (routing param, Part F4):

```
import { useParams, useNavigate } from "react-router-dom";

function TaskDetail() {
  const { id } = useParams();
  const navigate = useNavigate();
  const { tasks } = useTasks();
  const task = tasks.find(t => t.id === id);          // find (Phase 4!)

  if (!task) return <p className="p-4">Task not found.</p>;  // guard

  return (
    <div className="max-w-lg mx-auto p-4">
      <button onClick={() => navigate(-1)} className="text-blue-500 mb-4">< Back</button>
      <h1 className="text-2xl font-bold">{task.title}</h1>
      <p className="text-slate-500">Status: {task.done ? "Done 🟢" : "Active"}</p>
    </div>
  );
}
```

The lessons hiding in this app — every one is a concept you now own:

- **State → render, everywhere.** You never touch the DOM. `dispatch` changes the tasks array; React redraws. This is the Phase 4 `save(); render();` engine, fully automated — you predicted this a phase ago.
- **One source of truth.** `tasks` lives in one reducer, in one context. The list, the filter count, the detail page all read the *same* array. No syncing bugs.
- **Immutability is load-bearing.** Every reducer case returns a *new* array (`map`, `filter`, spread). That's what lets React detect changes and what makes the undo/redo/time-travel features possible later.
- **Composition of small pieces.** `TasksPage` → `FilterBar` + `TaskList` → `TaskItem`. Each does one job (Part C5).
- **Hooks compose.** `useLocalStorage` feeds `useReducer`; `useTasks` wraps `useContext`. Small tools, stacked.

Ship it (roadmap deliverable): `npm run build`, push to GitHub, deploy free on **Vercel**, put the live URL in your README with a screenshot. A deployed link is worth ten "I watched a tutorial"s.

Part M — Mindset: Thinking in React

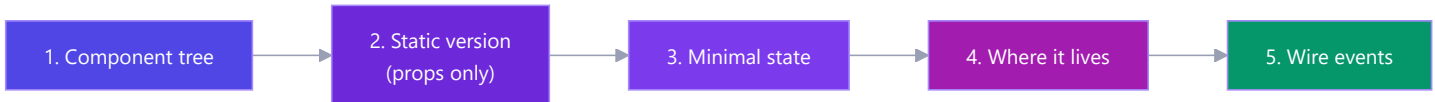
Syntax you can look up. The skill that makes you a React developer is knowing how to turn a picture in your head (or a Figma mockup) into components, props, and state. This is that skill, made into steps.

| M1. The Five Steps (the React docs' own method)

Simple definition: a repeatable process for building any React feature: break the UI into components, build a static version, find the minimal state, decide where it lives, and wire up the changes.

Telugu: Ee 5 steps ni React docs ye chetipoyi — **ఏ feature_రైనా** ee vrusalo kethochu. (1) UI ni components ga vidigolthi. (2) modat static version kethi (state lekunda, kethalo props). (3) **kani state** entho kunnokko. (4) aa state ekkada undalo nisthoyinchu. (5) maarpulu (events) ni wire cheyi. Phase 4 loni "understand → decompose → dumb version → refine" — adhi, React version.

1. **Break the UI into a component tree.** Draw boxes around the mockup; each box is a component. Spot repetition → that's a component used via `.map` (Part C5).
2. **Build a static version first.** Hard-code the data as props, no state, no interactivity — just get it *looking* right. This is the "dumbest version that works" from Phase 4 L4.
3. **Find the minimal state.** List every piece of data. Then ask of each: *does it change over time? is it passed in from a parent? can I compute it from other state?* If yes to any, it's **not** state. What's left is your minimal state.
4. **Decide where state lives.** For each piece, find the *lowest common parent* of every component that uses it, and put it there (lifting state up, D5). If many far-apart components need it, Context (Part G).
5. **Wire up the changes.** Add the event handlers that call setters/dispatch. Data flows down as props; events flow up as function calls (Part C2).



M2. The State Questions (what should be state?)

Simple definition: the single most common beginner mistake is putting too much in state — storing values you could *compute*. Minimal state means fewer bugs.

Telugu: Beginners chetane **#1 tippu: state lo chala prethadam** — lekunchagilagi vishayalaniki kada state lo dachadam. Rule: **compute chetayagilagithe, state vaddu**. Example: `tasks` state; kani "done tasks count" state kadu — `tasks.filter(t => t.done).length` tho lekunchochu. Tskuv state = tskuv bugs = rendu viluvulu clash aye avasam idu.

Ask three questions. If any is "yes," it is NOT state:

Question	If yes...
Does a parent pass it in via props?	it's a prop , not state
Does it stay the same over time?	it's a constant
Can you compute it from existing state/props?	it's a derived value — compute it in render

The derived-value rule — compute, don't store:

```
// ❌ storing what you can compute → two sources of truth, easy to desync
const [tasks, setTasks] = useState([]);
const [activeCount, setActiveCount] = useState(0); // BUG WAITING TO HAPPEN

// ✅ compute it every render – always correct, one source of truth
const [tasks, setTasks] = useState([]);
const activeCount = tasks.filter(t => !t.done).length; // derived
```

Storing `activeCount` means every task change must *also* remember to update the count — miss one and they disagree. Computing it means it's *always* right. **When in doubt, derive it.**

M3. Debugging React & The Practice Loop

Telugu: React lo bug vst panic v^{du} — విధానం undi. (1) **React DevTools** install ch^{esi}, component tree lo props/state **నిజ విలువలు** ch^{ad}u. (2) "screen t^{pu}" ante "state t^{pu}" — state ni modat^o ch^{ad}u, DOM ni k^{adu}. (3) `console.log` ni render i^o p^orti, e value e render i^o e^{mu}nd^o ch^{ad}u. Phase 4 L8 loni daily loop ikk^{ad} k^{ad}a — **type, break, fix, push.**

The React debugging method:

- **Install React DevTools** (browser extension). Its Components tab shows the live tree with each component's *actual* props and state — the first place to look when something's wrong.
- **"Wrong screen" = "wrong state."** Because the screen is a function of state, a visual bug is a *state* bug. Inspect the state first; the DOM is just its reflection.
- `console.log` **in the render body** shows you what value each render sees — invaluable for stale-state and dependency bugs.
- **Read the error overlay.** React's dev errors are unusually good — "Cannot read property of undefined" usually means data hasn't loaded yet (add a loading guard, B4).

Common bugs and their one-line fixes:

Symptom	Usual cause	Fix
"Each child needs a key" warning	missing/duplicate <code>key</code>	stable <code>key={item.id}</code> (B5)
Screen won't update on change	mutated state	immutable update (D4)
Infinite re-render loop	<code>setState</code> in render / bad deps	move to effect / event, fix deps (E5)
Stale value in a handler	snapshot closure	updater form <code>setX(x => ...)</code> (D3)
"Too many re-renders"	calling a setter during render	wrap it in a handler <code>onClick={() => ...}</code>

The daily practice loop (Phase 4 L8, React edition — 45 min):

1. **Recall (5 min):** from memory, "what does `useEffect`'s dependency array do?" Retrieval beats re-reading.
2. **Learn one thing (15 min):** one section of these notes — definition, Telugu line, the diagram.

3. **Type & break it (15 min):** never paste. Then break it on purpose — remove a dependency, mutate state directly, drop a `key`. Predict the bug *before* running.
4. **Explain (5 min):** out loud, "why does `useCallback` need `React.memo` to matter?" The sentence you can't finish is tomorrow's topic.
5. **Log it (5 min):** one line in `learnings.md`, push it.

The mindset, in one line (carried from Phase 4): *Don't try to be smart enough to hold the whole app in your head. Be systematic enough that you don't have to.* Break into components → static first → minimal state → place it → wire events. Every feature, every time. React isn't hard — it's a small set of ideas (state → render, data down, events up) applied relentlessly.

Part N — Exercises & Mini-Projects

Do these in order — each maps to a day of your roadmap. Every one is solvable with only these notes. Push each to GitHub; that's your Phase 6 trail. Build in Vite, style with Tailwind.

Day 1 — JSX, Components & Props

1. Build **five components**: `Button`, `Card`, `Badge`, `Avatar`, `Alert`. Pass props, give defaults, style with Tailwind. (Roadmap Day 1 deliverable.)
2. Make `Badge` accept a `tone` prop (`info` / `success` / `warn` / `error`) and pick its classes from a style map (K6).
3. Render an array of 5 products with `.map` and a proper `key`. Add conditional rendering: show a "Sold out" badge when `stock === 0`.
4. Build a `<Card>` that uses `children` so you can nest any content inside it (C4).

Day 2 — `useState`, `useEffect` & Forms

5. Build a **counter** with `+/-/reset`. Then break it: call `setCount(count+1)` three times in one handler and observe you get `+1`. Fix it with the updater form (D3).
6. Build a **controlled form** (name, email, role select) with one `handleChange` keyed by `name` (D6). Log the object on submit; clear the form after.
7. **[Roadmap]** Fetch users from `https://jsonplaceholder.typicode.com/users` with `useEffect`; show **loading / error / success** states (E4).
8. Build a live **character counter** textarea (280 limit) that turns red past the limit.
9. Build a `Clock` with `setInterval` in an effect — then *remove* the cleanup and watch what breaks on re-mount (E3).

Day 3 — React Router v6

10. **[Roadmap]** Build a **3-page app**: Home, Posts list, Post detail. Use `useParams` for the detail, a `*` 404 route, and a "Back" button with `useNavigate` (F).
11. Add a `NavLink` navbar that highlights the active tab (F3).
12. Wrap the pages in a `<Layout>` with a shared navbar + `<Outlet/>` (F6).

Day 4 — Context & Custom Hooks

13. Build a `ThemeContext` (light/dark) with a `useTheme()` hook; toggle the `dark` class on `<html>` (G3, K4).
14. Build a mock `AuthContext` with `user`, `login`, `logout`; make a navbar that shows Login or Logout accordingly (G4).
15. Write the three custom hooks from memory: `useFetch`, `useLocalStorage`, `useDebounce` (H2–H4). *This is the break-day test — write each from a blank file.*
16. **[Roadmap Q1]** Build a debounced search input (500ms) using your `useDebounce` + `useFetch`.
17. **[Roadmap Q2]** Build `useLocalStorage(key, initial)` and prove a value survives refresh.

Day 5 — useReducer & Tailwind

18. **[Roadmap Q3]** Build a **shopping cart** with `useReducer`: `ADD_ITEM`, `REMOVE_ITEM`, `UPDATE_QTY`, `CLEAR_CART` (I4). Compute the total with `reduce`.
19. **[Roadmap]** Rewrite the Day 2 form using `useReducer` instead of multiple `useState`s.
20. Style **5 components with only Tailwind** (no custom CSS). Build a **responsive card grid** `grid-cols-1 md:grid-cols-2 lg:grid-cols-3` (K3).
21. Add **dark mode** to all five via `dark:` variants wired to your `ThemeContext`.

Day 6 — Capstone

22. **[Roadmap Q4]** Build a `ProtectedRoute` that redirects guests to `/login` (F7 + `AuthContext`).
23. **[Roadmap Q5]** Write the answer, in your own words: *when does a `useEffect` cleanup run?*
24. **Ship the Task Tracker** (Part L): add/edit/delete, filter (All/Active/Done), `useLocalStorage`, dark theme via Context, Router `/tasks` + `/tasks/:id`. **Deploy on Vercel**, live URL in the README.

Mini-projects (pick 2, ship to GitHub — roadmap ideas)

- **Task Tracker App** — the Part L capstone, deployed. *(Phase 6 primary deliverable.)*
- **Weather Dashboard** — `useFetch` a free weather API, multiple saved cities via Context, dark-mode toggle. `??` defaults for missing fields (Phase 4 C3).
- **Component Library Sketch** — 5 reusable Tailwind components (Button, Card, Modal, Toast, Dropdown) + a demo page. `children` composition throughout.

Part O — Resources & Quick Reference

Learn more (matched to your roadmap)

- **React official docs** — the new docs are excellent and interactive; "Quick Start" then "Learn": <https://react.dev/learn>
- **Tailwind docs** — searchable, with a class reference you'll keep open forever: <https://tailwindcss.com/docs>
- **React Router docs** — the v6 tutorial: <https://reactrouter.com/en/main/start/tutorial>
- **Net Ninja — React 18 full playlist** (your roadmap's main watch): <https://youtu.be/j942wKiXFu8>
- **Dave Gray — Tailwind CSS Full Course**: <https://youtu.be/ICxcTsOHrjo>

- **Traversy — React Router v6:** <https://youtu.be/oTlJunBa6MA> · **Web Dev Simplified — Custom Hooks:** <https://youtu.be/6ThXsUwLWvc>
- **Web Dev Simplified — useState / useEffect** short, focused explainers: <https://youtu.be/O6P86uwfdR0>
- **Certifications:** Meta *React Basics* (Coursera, free audit); freeCodeCamp *Front End Development Libraries* (free cert); HackerRank *React (Basic)* badge.

How Phase 6 connects to your journey

- **From Phase 4/5:** `useState` is array destructuring; immutable updates *are* spread/map/filter; `useEffect` cleanup is subscribe/unsubscribe; the whole thing is your hand-built `save(); render();`. React added no new JavaScript — it organised the JS you already know.
- **To SAP Fiori/UI5 (later phases):** components, props/data-binding, routing, and centralised models all map across. React makes UI5 feel familiar, not foreign — which is exactly why it's in your roadmap.
- **Next up (Phase 7, Node.js):** the `fetch` calls your React app makes will hit APIs *you* build. Same JSON, other end of the wire.

One-page recall card

Component: a function returning JSX, name Capitalised · used like `<Button/>` · **JSX:** one root · `className` not `class` · close every tag · `{expr}` for JS · camelCase attrs. **Conditional:** `cond ? A : B` · `cond &&` `<UI/>` (beware `0 !`) · early `return` for whole branches. **Lists:** `.map` → JSX · stable `key={item.id}` · never index, never `Math.random()` · **Props:** data **down**, read-only · destructure in params · events flow **up** via function props · `children` for composition. **State:** `const [x, setX] = useState(init)` · never assign directly · state is a **snapshot** → `setX(x => ...)` when it depends on prev. **Immutable updates:** `{...obj, k:v}` · `[...arr, x]` · `.map` / `.filter` — never `push` / mutate. **Controlled form:** `value={s}` + `onChange={e => setS(e.target.value)}` · `e.preventDefault()` · **useEffect:** runs after render · `[]` once · `[dep]` on change · **return cleanup** for timers/listeners · async fn *inside*, not the effect itself. **Fetch shape:** loading / error / data · check `res.ok` · try/catch/finally · guard races. **Router:** `<BrowserRouter><Routes>` `<Route path element/></Routes>` · `<Link>` not `<a>` · `:id` + `useParams` · `useNavigate` in code · `<Outlet/>` layouts · `* 404` · `<Navigate>` guards. **Context:** `createContext` → `<Provider value>` → `useContext` · kills prop drilling · for global-ish, low-churn data (theme, auth). **Custom hook:** `use`-prefixed fn calling hooks · shares *logic*, not *state* · `useFetch` / `useLocalStorage` / `useDebounce` · **useReducer:** `[state, dispatch] = useReducer(reducer, init)` · reducer is pure `(state, action) => newState` · dispatch *what happened*. **Rules of Hooks:** top level only (no if/loop) · React functions only. **Perf (measure first):** `React.memo` (skip render) + `useCallback` (stable fn) + `useMemo` (cache value) — a team, used surgically. **useRef:** mutable box, no re-render · DOM refs + timer ids. **Tailwind:** utility-first · `p-4 bg-blue-500 rounded-lg` · mobile-first `md: lg:` · states `hover: focus: dark:` · repeat? → extract a **component**. **The one idea:** you never touch the DOM. Change **state**, React renders. Data down, events up. That's all of it.

Phase 6 turns everything you learned into things people can see and click. You built the `state → render` engine by hand in Phase 4; React just handed you the industrial version. Now: `npm create vite`, type every snippet, break it, fix it, deploy it. On to the Task Tracker.